QUICK-FIXING MISSING METADATA

If you're using the Spring Tool Suite, there's a quick-fix option for creating missing property metadata. Place your cursor on the line with the missing metadata warning and open the quick-fix pop-up with CMD-1 on Mac or Ctrl-1 on Windows and Linux (see figure 6.4).

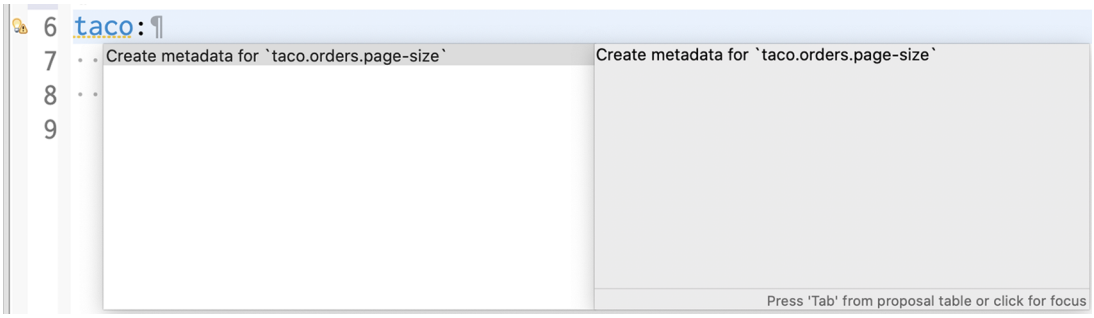

Figure 6.4 Creating configuration property metadata with the quick-fix pop-up in Spring Tool Suite

Then select the "Create Metadata for …" option to add some metadata for the property. If it doesn't already exist, this quick fix will create a file in META-INF/additional-spring-configuration-metadata.json and fill it in with some metadata for the `pageSize` property, as shown in the next code:

```
{"properties": [{
  "name": "taco.orders.page-size",
  "type": "java.lang.String",
  "description": "A description for 'taco.orders.page-size'"
}]}
```

Notice that the property name referenced in the metadata is `taco.orders.page-size`, whereas the actual property name in application.yml is `pageSize`. Spring Boot's flexible property naming allows for variations in property names such that `taco.orders.page-size` is equivalent to `taco.orders.pageSize`, so it doesn't matter much which form you use.

The initial metadata written to additional-spring-configuration-metadata.json is a fine start, but you'll probably want to edit it a little. Firstly, the `pageSize` property isn't a `java.lang.String`, so you'll want to change it to `java.lang.Integer`. And the `description` property should be changed to be more descriptive of what `pageSize` is for. The following JSON code sample shows what the metadata might look like after a few edits:

```
{"properties": [{
  "name": "taco.orders.page-size",
  "type": "java.lang.Integer",
```

```
  "description": "Sets the maximum number of orders to display in a list."
}]}
```

With that metadata in place, the warnings should be gone. What's more, if you hover over the `taco.orders.pageSize` property, you'll see the description shown in figure 6.5.

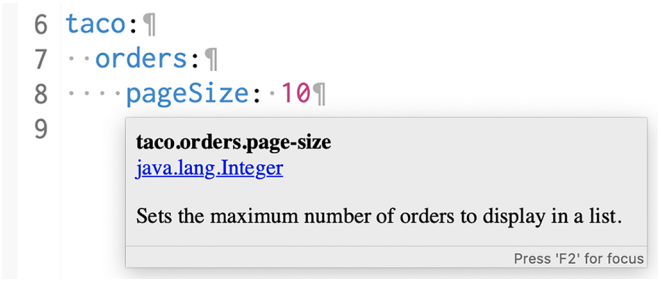

Figure 6.5 Hover help for custom configuration properties

Also, as shown in figure 6.6, you get autocompletion help from the IDE, just like Spring-provided configuration properties.

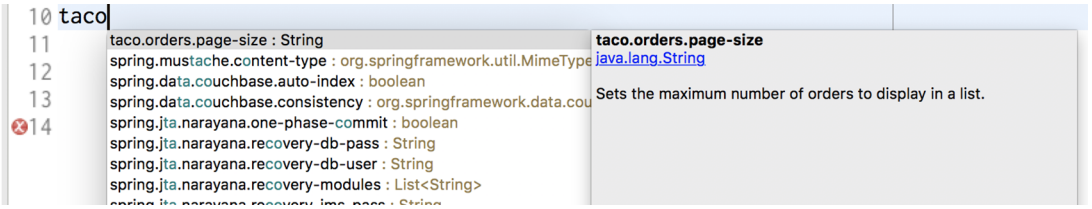

Figure 6.6 Configuration property metadata enables autocompletion of properties.

As you've seen, configuration properties are useful for tweaking both autoconfigured components as well as the details injected into your own application beans. But what if you need to configure different properties for different deployment environments? Let's take a look at how to use Spring profiles to set up environment-specific configurations.

6.3 Configuring with profiles

When applications are deployed to different runtime environments, usually some configuration details differ. The details of a database connection, for instance, are likely not the same in a development environment as in a quality assurance environment, and they are different still in a production environment. One way to configure properties uniquely in one environment over another is to use environment variables to specify configuration properties instead of defining them in application.properties and application.yml.

For instance, during development you can lean on the autoconfigured embedded H2 database. But in production, you can set database configuration properties as environment variables like this:

```
% export SPRING_DATASOURCE_URL=jdbc:mysql://localhost/tacocloud
% export SPRING_DATASOURCE_USERNAME=tacouser
% export SPRING_DATASOURCE_PASSWORD=tacopassword
```

Although this will work, it's somewhat cumbersome to specify more than one or two configuration properties as environment variables. Moreover, there's no good way to track changes to environment variables or to easily roll back changes if there's a mistake.

Instead, I prefer to take advantage of Spring profiles. Profiles are a type of conditional configuration where different beans, configuration classes, and configuration properties are applied or ignored based on what profiles are active at run time.

For instance, let's say that for development and debugging purposes, you want to use the embedded H2 database, and you want the logging levels for the Taco Cloud code to be set to `DEBUG`. But in production, you want to use an external MySQL database and set the logging levels to `WARN`. In the development situation, it's easy enough to not set any data source properties and get the autoconfigured H2 database. And as for debug-level logging, you can set the `logging.level.tacos` property for the `tacos` base package to `DEBUG` in application.yml as follows:

```
logging:
  level:
    tacos: DEBUG
```

This is precisely what you need for development purposes. But if you were to deploy this application in a production setting with no further changes to application.yml, you'd still have debug logging for the `tacos` package and an embedded H2 database. What you need is to define a profile with properties suited for production.

6.3.1 Defining profile-specific properties

One way to define profile-specific properties is to create yet another YAML or properties file containing only the properties for production. The name of the file should follow this convention: application-{profile name}.yml or application-{profile name}.properties. Then you can specify the configuration properties appropriate to that profile. For example, you could create a new file named application-prod.yml that contains the following properties:

```
spring:
  datasource:
    url: jdbc:mysql://localhost/tacocloud
    username: tacouser
    password: tacopassword
```

```
logging:
  level:
    tacos: WARN
```

Another way to specify profile-specific properties works only with YAML configuration. It involves placing profile-specific properties alongside nonprofiled properties in application.yml, separated by three hyphens and the `spring.profiles` property to name the profile. When applying the production properties to application.yml in this way, the entire application.yml would look like this:

```
logging:
  level:
    tacos: DEBUG

---
spring:
  profiles: prod

  datasource:
    url: jdbc:mysql://localhost/tacocloud
    username: tacouser
    password: tacopassword

logging:
  level:
    tacos: WARN
```

As you can see, this application.yml file is divided into two sections by a set of triple hyphens (`---`). The second section specifies a value for `spring.profiles`, indicating that the properties that follow apply to the `prod` profile. The first section, on the other hand, doesn't specify a value for `spring.profiles`. Therefore, its properties are common to all profiles or are defaults if the active profile doesn't otherwise have the properties set.

Regardless of which profiles are active when the application runs, the logging level for the `tacos` package will be set to `DEBUG` by the property set in the default profile. But if the profile named `prod` is active, then the `logging.level.tacos` property will be overridden with `WARN`. Likewise, if the `prod` profile is active, then the data source properties will be set to use the external MySQL database.

You can define properties for as many profiles as you need by creating additional YAML or properties files named with the pattern application-{profile name}.yml or application-{profile name}.properties. Or, if you prefer, type three more dashes in application.yml along with another `spring.profiles` property to specify the profile name. Then add all of the profile-specific properties you need. Although there's no benefit to either approach, you might find that putting all profile configurations in a single YAML file works best when the number of properties is small, whereas distinct files for each profile is better when you have a large number of properties.

6.3.2 Activating profiles

Setting profile-specific properties will do no good unless those profiles are active. But how can you make a profile active? All it takes to make a profile active is to include it in the list of profile names given to the `spring.profiles.active` property. For example, you could set it in application.yml like this:

```
spring:
  profiles:
    active:
    - prod
```

But that's perhaps the worst possible way to set an active profile. If you set the active profile in application.yml, then that profile becomes the default profile, and you achieve none of the benefits of using profiles to separate the production-specific properties from development properties. Instead, I recommend that you set the active profile(s) with environment variables. On the production environment, you would set `SPRING_PROFILES_ACTIVE` like this:

```
% export SPRING_PROFILES_ACTIVE=prod
```

From then on, any applications deployed to that machine will have the `prod` profile active, and the corresponding configuration properties would take precedence over the properties in the default profile.

If you're running the application as an executable JAR file, you might also set the active profile with a command-line argument like this:

```
% java -jar taco-cloud.jar --spring.profiles.active=prod
```

Note that the `spring.profiles.active` property name contains the plural word *profiles*. This means you can specify more than one active profile. Often, this is with a comma-separated list as when setting it with an environment variable, as shown here:

```
% export SPRING_PROFILES_ACTIVE=prod,audit,ha
```

But in YAML, you'd specify it as a list like this:

```
spring:
  profiles:
    active:
    - prod
    - audit
    - ha
```

It's also worth noting that if you deploy a Spring application to Cloud Foundry, a profile named `cloud` is automatically activated for you. If Cloud Foundry is your production environment, you'll want to be sure to specify production-specific properties under the `cloud` profile.

As it turns out, profiles aren't useful only for conditionally setting configuration properties in a Spring application. Let's see how to declare beans specific to an active profile.

6.3.3 Conditionally creating beans with profiles

Sometimes it's useful to provide a unique set of beans for different profiles. Normally, any bean declared in a Java configuration class is created, regardless of which profile is active. But suppose you need some beans to be created only if a certain profile is active. In that case, the `@Profile` annotation can designate beans as being applicable to only a given profile.

For instance, suppose you have a `CommandLineRunner` bean declared in `TacoCloudApplication` that's used to load the embedded database with ingredient data when the application starts. That's great for development but would be unnecessary (and undesirable) in a production application. To prevent the ingredient data from being loaded every time the application starts in a production deployment, you could annotate the `CommandLineRunner` bean method with `@Profile` like this:

```
@Bean
@Profile("dev")
public CommandLineRunner dataLoader(IngredientRepository repo,
      UserRepository userRepo, PasswordEncoder encoder) {

  ...

}
```

Or suppose that you need the `CommandLineRunner` created if either the `dev` profile or `qa` profile is active. In that case, you can list the profiles for which the bean should be created like so:

```
@Bean
@Profile({"dev", "qa"})
public CommandLineRunner dataLoader(IngredientRepository repo,
      UserRepository userRepo, PasswordEncoder encoder) {

  ...

}
```

Now the ingredient data will be loaded only if the `dev` or `qa` profiles are active. That would mean that you'd need to activate the `dev` profile when running the application in the development environment. It would be even more convenient if that `CommandLineRunner` bean were always created unless the `prod` profile is active. In that case, you can apply `@Profile` like this:

```
@Bean
@Profile("!prod")
```

```
public CommandLineRunner dataLoader(IngredientRepository repo,
      UserRepository userRepo, PasswordEncoder encoder) {

  ...

}
```

Here, the exclamation mark (`!`) negates the profile name. Effectively, it states that the `CommandLineRunner` bean will be created if the `prod` profile isn't active.

It's also possible to use `@Profile` on an entire `@Configuration`-annotated class. For example, suppose that you were to extract the `CommandLineRunner` bean into a separate configuration class named `DevelopmentConfig`. Then you could annotate `DevelopmentConfig` with `@Profile` as follows:

```
@Profile({"!prod", "!qa"})
@Configuration
public class DevelopmentConfig {

  @Bean
  public CommandLineRunner dataLoader(IngredientRepository repo,
        UserRepository userRepo, PasswordEncoder encoder) {

    ...

  }

}
```

Here, the `CommandLineRunner` bean (as well as any other beans defined in `DevelopmentConfig`) will be created only if neither the `prod` nor `qa` profile is active.

Summary

- We can annotate Spring beans with `@ConfigurationProperties` to enable injection of values from one of several property sources.
- Configuration properties can be set in command-line arguments, environment variables, JVM system properties, properties files, or YAML files, among other options.
- Use configuration properties to override autoconfiguration settings, including the ability to specify a data source URL and logging levels.
- Spring profiles can be used with property sources to conditionally set configuration properties based on the active profile(s).

Part 2

Integrated Spring

The chapters in part 2 cover topics that help integrate your Spring application with other applications.

Chapter 7 expands on the discussion of Spring MVC started in chapter 2 by looking at how to write REST APIs in Spring. We'll look at how to define REST endpoints in Spring MVC, automatically generate repository-based REST endpoints with Spring Data REST, and consume REST APIs. Chapter 8 looks at how to secure an API using Spring Security's support for OAuth 2 as well as how to obtain authorization in client code that can access OAuth 2–secured APIs. In chapter 9, we'll look at using asynchronous communication to enable a Spring application to both send and receive messages using the Java Message Service (JMS), RabbitMQ, and Kafka. And finally, chapter 10 discusses declarative application integration using the Spring Integration project. We'll cover processing data in real time, defining integration flows, and integrating with external systems like emails and filesystems.

7 Creating REST services

This chapter covers

- Defining REST endpoints in Spring MVC
- Automatic repository-based REST endpoints
- Consuming REST APIs

"The web browser is dead. What now?"

Several years ago, I heard someone suggest that the web browser was nearing legacy status and that something else would take over. But how could this be? What could possibly dethrone the near-ubiquitous web browser? How would we consume the growing number of sites and online services if not with a web browser? Surely these were the ramblings of a madman!

Fast-forward to the present day, and it's clear that the web browser hasn't gone away. But it no longer reigns as the primary means of accessing the internet. Mobile devices, tablets, smart watches, and voice-based devices are now commonplace. And even many browser-based applications are actually running JavaScript applications rather than letting the browser be a dumb terminal for server-rendered content.

With such a vast selection of client-side options, many applications have adopted a common design where the user interface is pushed closer to the client and the server exposes an API through which all kinds of clients can interact with the back-end functionality.

In this chapter, you're going to use Spring to provide a REST API for the Taco Cloud application. You'll use what you learned about Spring MVC in chapter 2 to create RESTful endpoints with Spring MVC controllers. You'll also automatically expose REST endpoints for the Spring Data repositories you defined in chapters 3 and 4. Finally, we'll look at ways to test and secure those endpoints.

But first, you'll start by writing a few new Spring MVC controllers that expose backend functionality with REST endpoints to be consumed by a rich web frontend.

7.1 Writing RESTful controllers

In a nutshell, REST APIs aren't much different from websites. Both involve responding to HTTP requests. But the key difference is that instead of responding to those requests with HTML, as websites do, REST APIs typically respond with a data-oriented format such as JSON or XML.

In chapter 2 you used `@GetMapping` and `@PostMapping` annotations to fetch and post data to the server. Those same annotations will still come in handy as you define your REST API. In addition, Spring MVC supports a handful of other annotations for various types of HTTP requests, as listed in table 7.1.

Table 7.1 Spring MVC's HTTP request-handling annotations

Annotation	HTTP method	Typical use[a]
`@GetMapping`	HTTP `GET` requests	Reading resource data
`@PostMapping`	HTTP `POST` requests	Creating a resource
`@PutMapping`	HTTP `PUT` requests	Updating a resource
`@PatchMapping`	HTTP `PATCH` requests	Updating a resource
`@DeleteMapping`	HTTP `DELETE` requests	Deleting a resource
`@RequestMapping`	General-purpose request handling; HTTP `method` specified in the method attribute	

a. Mapping HTTP methods to create, read, update, and delete (CRUD) operations isn't a perfect match, but in practice, that's how they're often used and how you'll use them in Taco Cloud.

To see these annotations in action, you'll start by creating a simple REST endpoint that fetches a few of the most recently created tacos.

7.1.1 Retrieving data from the server

One thing that we'd like the Taco Cloud application to be able to do is allow taco fanatics to design their own taco creations and share them with their follow taco lovers. One way to do that is to display a list of the most recently created tacos on the website.

In support of that feature, we need to create an endpoint that handles `GET` requests for /api/tacos which include a "recent" parameter and responds with a list of recently

designed tacos. You'll create a new controller to handle such a request. The next listing shows the controller for the job.

Listing 7.1 A RESTful controller for taco design API requests

```
package tacos.web.api;

import org.springframework.data.domain.PageRequest;
import org.springframework.data.domain.Sort;
import org.springframework.web.bind.annotation.CrossOrigin;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import tacos.Taco;
import tacos.data.TacoRepository;

@RestController
@RequestMapping(path="/api/tacos",
                produces="application/json")
@CrossOrigin(origins="http://tacocloud:8080")
public class TacoController {
  private TacoRepository tacoRepo;

  public TacoController(TacoRepository tacoRepo) {
    this.tacoRepo = tacoRepo;
  }

  @GetMapping(params="recent")
  public Iterable<Taco> recentTacos() {
    PageRequest page = PageRequest.of(
            0, 12, Sort.by("createdAt").descending());
    return tacoRepo.findAll(page).getContent();
  }
}
```

Handles requests for /api/tacos

Allows cross-origin requests

Fetches and returns recent taco designs

You may be thinking that this controller's name sounds somewhat familiar. In chapter 2 you created a similarly named `DesignTacoController` that handled similar types of requests. But where that controller was for producing an HTML result in the Taco Cloud application, this new `TacoController` is a REST controller, as indicated by the `@RestController` annotation.

The `@RestController` annotation serves two purposes. First, it's a stereotype annotation like `@Controller` and `@Service` that marks a class for discovery by component scanning. But most relevant to the discussion of REST, the `@RestController` annotation tells Spring that all handler methods in the controller should have their return value written directly to the body of the response, rather than being carried in the model to a view for rendering.

Alternatively, you could have annotated `TacoController` with `@Controller`, just like any Spring MVC controller. But then you'd need to also annotate all of the handler

methods with `@ResponseBody` to achieve the same result. Yet another option would be to return a `ResponseEntity` object, which we'll discuss in a moment.

The `@RequestMapping` annotation at the class level works with the `@GetMapping` annotation on the `recentTacos()` method to specify that the `recentTacos()` method is responsible for handling `GET` requests for /design?recent.

You'll notice that the `@RequestMapping` annotation also sets a `produces` attribute. This specifies that any of the handler methods in `TacoController` will handle requests only if the client sends a request with an `Accept` header that includes `"application/json"`, indicating that the client can handle responses only in JSON format. This use of `produces` limits your API to only producing JSON results, and it allows for another controller (perhaps the `TacoController` from chapter 2) to handle requests with the same paths, so long as those requests don't require JSON output.

Even though setting `produces` to `"application/json"` limits your API to being JSON-based (which is fine for your needs), you're welcome to set `produces` to an array of `String` for multiple content types. For example, to allow for XML output, you could add `"text/xml"` to the `produces` attribute as follows:

```
@RequestMapping(path="/api/tacos",
                produces={"application/json", "text/xml"})
```

The other thing you may have noticed in listing 7.1 is that the class is annotated with `@CrossOrigin`. It's common for a JavaScript-based user interface, such as those written in a framework like Angular or ReactJS, to be served from a separate host and/or port from the API (at least for now), and the web browser will prevent your client from consuming the API. This restriction can be overcome by including CORS (cross-origin resource sharing) headers in the server responses. Spring makes it easy to apply CORS with the `@CrossOrigin` annotation.

As applied here, `@CrossOrigin` allows clients from localhost, port 8080, to access the API. The `origins` attribute accepts an array, however, so you can also specify multiple values, as shown next:

```
@RestController
@RequestMapping(path="/api/tacos",
                produces="application/json")
@CrossOrigin(origins={"http://tacocloud:8080", "http://tacocloud.com"})
public class TacoController {
  ...
}
```

The logic within the `recentTacos()` method is fairly straightforward. It constructs a `PageRequest` object that specifies that you want only the first (0th) page of 12 results, sorted in descending order by the taco's creation date. In short, you want a dozen of the most recently created taco designs. The `PageRequest` is passed into the call to the `findAll()` method of `TacoRepository`, and the content of that page

of results is returned to the client (which, as you saw in listing 7.1, will be used as model data to display to the user).

You now have the start of a Taco Cloud API for your client. For development testing purposes, you may also want to use command-line utilities like `curl` or HTTPie (https://httpie.org/) to poke about the API. For example, the following command line shows how you might fetch recently created tacos with `curl`:

```
$ curl localhost:8080/api/tacos?recent
```

Or like this, if you prefer HTTPie:

```
$ http :8080/api/tacos?recent
```

Initially, the database will be empty, so the results from these requests will likewise be empty. We'll see in a moment how to handle `POST` requests that save tacos. But in the meantime, you could add an `CommandLineRunner` bean to preload the database with some test data. The following `CommandLineRunner` bean method shows how you might preload a few ingredients and a few tacos:

```
@Bean
public CommandLineRunner dataLoader(
    IngredientRepository repo,
    UserRepository userRepo,
    PasswordEncoder encoder,
    TacoRepository tacoRepo) {
  return args -> {
    Ingredient flourTortilla = new Ingredient(
        "FLTO", "Flour Tortilla", Type.WRAP);
    Ingredient cornTortilla = new Ingredient(
        "COTO", "Corn Tortilla", Type.WRAP);
    Ingredient groundBeef = new Ingredient(
        "GRBF", "Ground Beef", Type.PROTEIN);
    Ingredient carnitas = new Ingredient(
        "CARN", "Carnitas", Type.PROTEIN);
    Ingredient tomatoes = new Ingredient(
        "TMTO", "Diced Tomatoes", Type.VEGGIES);
    Ingredient lettuce = new Ingredient(
        "LETC", "Lettuce", Type.VEGGIES);
    Ingredient cheddar = new Ingredient(
        "CHED", "Cheddar", Type.CHEESE);
    Ingredient jack = new Ingredient(
        "JACK", "Monterrey Jack", Type.CHEESE);
    Ingredient salsa = new Ingredient(
        "SLSA", "Salsa", Type.SAUCE);
    Ingredient sourCream = new Ingredient(
        "SRCR", "Sour Cream", Type.SAUCE);
    repo.save(flourTortilla);
    repo.save(cornTortilla);
    repo.save(groundBeef);
    repo.save(carnitas);
    repo.save(tomatoes);
```

```
    repo.save(lettuce);
    repo.save(cheddar);
    repo.save(jack);
    repo.save(salsa);
    repo.save(sourCream);

    Taco taco1 = new Taco();
    taco1.setName("Carnivore");
    taco1.setIngredients(Arrays.asList(
            flourTortilla, groundBeef, carnitas,
            sourCream, salsa, cheddar));
    tacoRepo.save(taco1);

    Taco taco2 = new Taco();
    taco2.setName("Bovine Bounty");
    taco2.setIngredients(Arrays.asList(
            cornTortilla, groundBeef, cheddar,
            jack, sourCream));
    tacoRepo.save(taco2);

    Taco taco3 = new Taco();
    taco3.setName("Veg-Out");
    taco3.setIngredients(Arrays.asList(
            flourTortilla, cornTortilla, tomatoes,
            lettuce, salsa));
    tacoRepo.save(taco3);
  };
}
```

Now if you try to use `curl` or HTTPie to make a request to the recent tacos endpoint, you'll get a response something like this (response formatted for readability):

```
$ curl localhost:8080/api/tacos?recent
[
  {
    "id": 4,
    "name": "Veg-Out",
    "createdAt": "2021-08-02T00:47:09.624+00:00",
    "ingredients": [
      { "id": "FLTO", "name": "Flour Tortilla", "type": "WRAP" },
      { "id": "COTO", "name": "Corn Tortilla", "type": "WRAP" },
      { "id": "TMTO", "name": "Diced Tomatoes", "type": "VEGGIES" },
      { "id": "LETC", "name": "Lettuce", "type": "VEGGIES" },
      { "id": "SLSA", "name": "Salsa", "type": "SAUCE" }
    ]
  },
  {
    "id": 3,
    "name": "Bovine Bounty",
    "createdAt": "2021-08-02T00:47:09.621+00:00",
    "ingredients": [
      { "id": "COTO", "name": "Corn Tortilla", "type": "WRAP" },
      { "id": "GRBF", "name": "Ground Beef", "type": "PROTEIN" },
      { "id": "CHED", "name": "Cheddar", "type": "CHEESE" },
```

```
      { "id": "JACK", "name": "Monterrey Jack", "type": "CHEESE" },
      { "id": "SRCR", "name": "Sour Cream", "type": "SAUCE" }
    ]
  },
  {
    "id": 2,
    "name": "Carnivore",
    "createdAt": "2021-08-02T00:47:09.520+00:00",
    "ingredients": [
      { "id": "FLTO", "name": "Flour Tortilla", "type": "WRAP" },
      { "id": "GRBF", "name": "Ground Beef", "type": "PROTEIN" },
      { "id": "CARN", "name": "Carnitas", "type": "PROTEIN" },
      { "id": "SRCR", "name": "Sour Cream", "type": "SAUCE" },
      { "id": "SLSA", "name": "Salsa", "type": "SAUCE" },
      { "id": "CHED", "name": "Cheddar", "type": "CHEESE" }
    ]
  }
]
```

Now let's say that you want to offer an endpoint that fetches a single taco by its ID. By using a placeholder variable in the handler method's path and accepting a path variable, you can capture the ID and use it to look up the `Taco` object through the repository as follows:

```
@GetMapping("/{id}")
public Optional<Taco> tacoById(@PathVariable("id") Long id) {
  return tacoRepo.findById(id);
}
```

Because the controller's base path is /api/tacos, this controller method handles `GET` requests for /api/tacos/{id}, where the {id} portion of the path is a placeholder. The actual value in the request is given to the `id` parameter, which is mapped to the `{id}` placeholder by `@PathVariable`.

Inside of `tacoById()`, the `id` parameter is passed to the repository's `findById()` method to fetch the `Taco`. The repository's `findById()` method returns an `Optional<Taco>`, because it is possible that there may not be a taco that matches the given ID. The `Optional<Taco>` is simply returned from the controller method.

Spring then takes the `Optional<Taco>` and calls its `get()` method to produce the response. If the ID doesn't match any known tacos, the response body will contain "null" and the response's HTTP status code will be 200 (OK). The client is handed a response it can't use, but the status code indicates everything is fine. A better approach would be to return a response with an HTTP 404 (NOT FOUND) status.

As it's currently written, there's no easy way to return a 404 status code from `tacoById()`. But if you make a few small tweaks, you can set the status code appropriately, as shown here:

```
@GetMapping("/{id}")
public ResponseEntity<Taco> tacoById(@PathVariable("id") Long id) {
  Optional<Taco> optTaco = tacoRepo.findById(id);
```

```
  if (optTaco.isPresent()) {
    return new ResponseEntity<>(optTaco.get(), HttpStatus.OK);
  }
  return new ResponseEntity<>(null, HttpStatus.NOT_FOUND);
}
```

Now, instead of returning a `Taco` object, `tacoById()` returns a `ResponseEntity<Taco>`. If the taco is found, you wrap the `Taco` object in a `ResponseEntity` with an HTTP status of OK (which is what the behavior was before). But if the taco isn't found, you wrap a `null` in a `ResponseEntity` along with an HTTP status of NOT FOUND to indicate that the client is trying to fetch a taco that doesn't exist.

Defining an endpoint that returns information is only the start. What if your API needs to receive data from the client? Let's see how you can write controller methods that handle input on the requests.

7.1.2 Sending data to the server

So far your API is able to return up to a dozen of the most recently created tacos. But how do those tacos get created in the first place?

Although you could use a `CommandLineRunner` bean to preload the database with some test taco data, ultimately taco data will come from users when they craft their taco creations. Therefore, we'll need to write a method in `TacoController` that handles requests containing taco designs and save them to the database. By adding the following `postTaco()` method to `TacoController`, you enable the controller to do exactly that:

```
@PostMapping(consumes="application/json")
@ResponseStatus(HttpStatus.CREATED)
public Taco postTaco(@RequestBody Taco taco) {
  return tacoRepo.save(taco);
}
```

Because `postTaco()` will handle an HTTP `POST` request, it's annotated with `@PostMapping` instead of `@GetMapping`. You're not specifying a `path` attribute here, so the `postTaco()` method will handle requests for /api/tacos as specified in the class-level `@RequestMapping` on `TacoController`.

You do set the `consumes` attribute, however. The `consumes` attribute is to request input what `produces` is to request output. Here you use `consumes` to say that the method will only handle requests whose `Content-type` matches `application/json`.

The method's `Taco` parameter is annotated with `@RequestBody` to indicate that the body of the request should be converted to a `Taco` object and bound to the parameter. This annotation is important—without it, Spring MVC would assume that you want request parameters (either query parameters or form parameters) to be bound to the `Taco` object. But the `@RequestBody` annotation ensures that JSON in the request body is bound to the `Taco` object instead.

Once `postTaco()` has received the `Taco` object, it passes it to the `save()` method on the `TacoRepository`.

You may have also noticed that I've annotated the `postTaco()` method with `@ResponseStatus(HttpStatus.CREATED)`. Under normal circumstances (when no exceptions are thrown), all responses will have an HTTP status code of 200 (OK), indicating that the request was successful. Although an HTTP 200 response is always welcome, it's not always descriptive enough. In the case of a `POST` request, an HTTP status of 201 (CREATED) is more descriptive. It tells the client that not only was the request successful but a resource was created as a result. It's always a good idea to use `@ResponseStatus` where appropriate to communicate the most descriptive and accurate HTTP status code to the client.

Although you've used `@PostMapping` to create a new `Taco` resource, `POST` requests can also be used to update resources. Even so, `POST` requests are typically used for resource creation, and `PUT` and `PATCH` requests are used to update resources. Let's see how you can update data using `@PutMapping` and `@PatchMapping`.

7.1.3 Updating data on the server

Before you write any controller code for handling HTTP `PUT` or `PATCH` commands, you should take a moment to consider the elephant in the room: why are there two different HTTP methods for updating resources?

Although it's true that `PUT` is often used to update resource data, it's actually the semantic opposite of `GET`. Whereas `GET` requests are for transferring data from the server to the client, `PUT` requests are for sending data from the client to the server.

In that sense, `PUT` is really intended to perform a wholesale *replacement* operation rather than an update operation. In contrast, the purpose of HTTP `PATCH` is to perform a patch or partial update of resource data.

For example, suppose you want to be able to change the address on an order. One way we could achieve this through the REST API is with a `PUT` request handled like this:

```
@PutMapping(path="/{orderId}", consumes="application/json")
public TacoOrder putOrder(
                      @PathVariable("orderId") Long orderId,
                      @RequestBody TacoOrder order) {
  order.setId(orderId);
  return repo.save(order);
}
```

This could work, but it would require that the client submit the complete order data in the `PUT` request. Semantically, `PUT` means "put this data at this URL," essentially replacing any data that's already there. If any of the order's properties are omitted, that property's value would be overwritten with `null`. Even the tacos in the order would need to be set along with the order data or else they'd be removed from the order.

If `PUT` does a wholesale replacement of the resource data, then how should you handle requests to do just a partial update? That's what HTTP `PATCH` requests and

Spring's `@PatchMapping` are good for. Here's how you might write a controller method to handle a `PATCH` request for an order:

```
@PatchMapping(path="/{orderId}", consumes="application/json")
public TacoOrder patchOrder(@PathVariable("orderId") Long orderId,
                        @RequestBody TacoOrder patch) {

  TacoOrder order = repo.findById(orderId).get();
  if (patch.getDeliveryName() != null) {
    order.setDeliveryName(patch.getDeliveryName());
  }
  if (patch.getDeliveryStreet() != null) {
    order.setDeliveryStreet(patch.getDeliveryStreet());
  }
  if (patch.getDeliveryCity() != null) {
    order.setDeliveryCity(patch.getDeliveryCity());
  }
  if (patch.getDeliveryState() != null) {
    order.setDeliveryState(patch.getDeliveryState());
  }
  if (patch.getDeliveryZip() != null) {
    order.setDeliveryZip(patch.getDeliveryZip());
  }
  if (patch.getCcNumber() != null) {
    order.setCcNumber(patch.getCcNumber());
  }
  if (patch.getCcExpiration() != null) {
    order.setCcExpiration(patch.getCcExpiration());
  }
  if (patch.getCcCVV() != null) {
    order.setCcCVV(patch.getCcCVV());
  }
  return repo.save(order);
}
```

The first thing to note here is that the `patchOrder()` method is annotated with `@PatchMapping` instead of `@PutMapping`, indicating that it should handle HTTP `PATCH` requests instead of `PUT` requests.

But the one thing you've no doubt noticed is that the `patchOrder()` method is a bit more involved than the `putOrder()` method. That's because Spring MVC's mapping annotations, including `@PatchMapping` and `@PutMapping`, specify only what kinds of requests a method should handle. These annotations don't dictate how the request will be handled. Even though `PATCH` semantically implies a partial update, it's up to you to write code in the handler method that actually performs such an update.

In the case of the `putOrder()` method, you accepted the complete data for an order and saved it, adhering to the semantics of HTTP `PUT`. But in order for `patchMapping()` to adhere to the semantics of HTTP `PATCH`, the body of the method requires more intelligence. Instead of completely replacing the order with the new data sent in, it inspects each field of the incoming `TacoOrder` object and applies any non-`null` values to the existing order. This approach allows the client to send only the

properties that should be changed and enables the server to retain existing data for any properties not specified by the client.

> **There's more than one way to PATCH**
>
> The patching approach applied in the `patchOrder()` method has the following limitations:
>
> - If `null` values are meant to specify no change, how can the client indicate that a field should be set to `null`?
> - There's no way of removing or adding a subset of items from a collection. If the client wants to add or remove an entry from a collection, it must send the complete altered collection.
>
> There's really no hard-and-fast rule about how `PATCH` requests should be handled or what the incoming data should look like. Rather than sending the actual domain data, a client could send a patch-specific description of the changes to be applied. Of course, the request handler would have to be written to handle patch instructions instead of the domain data.

In both `@PutMapping` and `@PatchMapping`, notice that the request path references the resource that's to be changed. This is the same way paths are handled by `@GetMapping`-annotated methods.

You've now seen how to fetch and post resources with `@GetMapping` and `@PostMapping`. And you've seen two different ways of updating a resource with `@PutMapping` and `@PatchMapping`. All that's left is handling requests to delete a resource.

7.1.4 Deleting data from the server

Sometimes data simply isn't needed anymore. In those cases, a client should be able to request that a resource be removed with an HTTP `DELETE` request.

Spring MVC's `@DeleteMapping` comes in handy for declaring methods that handle `DELETE` requests. For example, let's say you want your API to allow for an order resource to be deleted. The following controller method should do the trick:

```
@DeleteMapping("/{orderId}")
@ResponseStatus(HttpStatus.NO_CONTENT)
public void deleteOrder(@PathVariable("orderId") Long orderId) {
  try {
    repo.deleteById(orderId);
  } catch (EmptyResultDataAccessException e) {}
}
```

By this point, the idea of another mapping annotation should be old hat to you. You've already seen `@GetMapping`, `@PostMapping`, `@PutMapping`, and `@PatchMapping`—each specifying that a method should handle requests for their corresponding HTTP methods. It will probably come as no surprise to you that `@DeleteMapping` is used to

specify that the `deleteOrder()` method is responsible for handling `DELETE` requests for /orders/{orderId}.

The code within the method is what does the actual work of deleting an order. In this case, it takes the order ID, provided as a path variable in the URL, and passes it to the repository's `deleteById()` method. If the order exists when that method is called, it will be deleted. If the order doesn't exist, an `EmptyResultDataAccessException` will be thrown.

I've chosen to catch the `EmptyResultDataAccessException` and do nothing with it. My thinking here is that if you try to delete a resource that doesn't exist, the outcome is the same as if it did exist prior to deletion—that is, the resource will be nonexistent. Whether it existed before is irrelevant. Alternatively, I could've written `deleteOrder()` to return a `ResponseEntity`, setting the body to `null` and the HTTP status code to NOT FOUND.

The only other thing to take note of in the `deleteOrder()` method is that it's annotated with `@ResponseStatus` to ensure that the response's HTTP status is 204 (NO CONTENT). There's no need to communicate any resource data back to the client for a resource that no longer exists, so responses to `DELETE` requests typically have no body and, therefore, should communicate an HTTP status code to let the client know not to expect any content.

Your Taco Cloud API is starting to take shape. Now a client can be written to consume this API, presenting ingredients, accepting orders, and displaying recently created tacos. We'll talk about writing REST client code a little later in 7.3. But for now, let's see another way to create REST API endpoints: automatically based on Spring Data repositories.

7.2 Enabling data-backed services

As you saw in chapter 3, Spring Data performs a special kind of magic by automatically creating repository implementations based on interfaces you define in your code. But Spring Data has another trick up its sleeve that can help you define APIs for your application.

Spring Data REST is another member of the Spring Data family that automatically creates REST APIs for repositories created by Spring Data. By doing little more than adding Spring Data REST to your build, you get an API with operations for each repository interface you've defined.

To start using Spring Data REST, add the following dependency to your build:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-rest</artifactId>
</dependency>
```

Believe it or not, that's all that's required to expose a REST API in a project that's already using Spring Data for automatic repositories. By simply having the Spring Data REST starter in the build, the application gets autoconfiguration that enables

automatic creation of a REST API for any repositories that were created by Spring Data (including Spring Data JPA, Spring Data Mongo, and so on).

The REST endpoints that Spring Data REST creates are at least as good as (and possibly even better than) the ones you've created yourself. So at this point, feel free to do a little demolition work and remove any `@RestController`-annotated classes you've created up to this point before moving on.

To try out the endpoints provided by Spring Data REST, you can fire up the application and start poking at some of the URLs. Based on the set of repositories you've already defined for Taco Cloud, you should be able to perform `GET` requests for tacos, ingredients, orders, and users.

For example, you can get a list of all ingredients by making a `GET` request for /ingredients. Using `curl`, you might get something that looks like this (abridged to show only the first ingredient):

```
$ curl localhost:8080/ingredients
{
  "_embedded" : {
    "ingredients" : [ {
      "name" : "Flour Tortilla",
      "type" : "WRAP",
      "_links" : {
        "self" : {
          "href" : "http://localhost:8080/ingredients/FLTO"
        },
        "ingredient" : {
          "href" : "http://localhost:8080/ingredients/FLTO"
        }
      }
    },
    ...
    ]
  },
  "_links" : {
    "self" : {
      "href" : "http://localhost:8080/ingredients"
    },
    "profile" : {
      "href" : "http://localhost:8080/profile/ingredients"
    }
  }
}
```

Wow! By doing nothing more than adding a dependency to your build, you're not only getting an endpoint for ingredients, but the resources that come back also contain hyperlinks! These hyperlinks are implementations of Hypermedia as the Engine of Application State, or HATEOAS for short. A client consuming this API could (optionally) use these hyperlinks as a guide for navigating the API and performing the next request.

The Spring HATEOAS project (https://spring.io/projects/spring-hateoas) provides general support for adding hypermedia links in your Spring MVC controller responses. But Spring Data REST automatically adds these links in the responses to its generated APIs.

> **To HATEOAS or not to HATEOAS?**
>
> The general idea of HATEOAS is that it enables a client to navigate an API in much the same way that a human may navigate a website: by following links. Rather than encode API details in a client and having the client construct URLs for every request, the client can select a link, by name, from the list of hyperlinks and use it to make their next request. In this way, the client doesn't need to be coded to know the structure of an API and can instead use the API itself as a roadmap through the API.
>
> On the other hand, the hyperlinks do add a small amount of extra data in the payload and add some complexity requiring that the client know how to navigate using those hyperlinks. For this reason, API developers often forego the use of HATEOAS, and client developers often simply ignore the hyperlinks if there are any in an API.
>
> Other than the free hyperlinks you get from Spring Data REST responses, we'll ignore HATEOAS and focus on simple, nonhypermedia APIs.

Pretending to be a client of this API, you can also use `curl` to follow the `self` link for the flour tortilla entry as follows:

```
$ curl http://localhost:8080/ingredients/FLTO
{
  "name" : "Flour Tortilla",
  "type" : "WRAP",
  "_links" : {
    "self" : {
      "href" : "http://localhost:8080/ingredients/FLTO"
    },
    "ingredient" : {
      "href" : "http://localhost:8080/ingredients/FLTO"
    }
  }
}
```

To avoid getting too distracted, we won't waste much more time in this book digging into each and every endpoint and option that Spring Data REST has created. But you should know that it also supports `POST`, `PUT`, and `DELETE` methods for the endpoints it creates. That's right: you can `POST` to /ingredients to create a new ingredient and `DELETE` /ingredients/FLTO to remove flour tortillas from the menu.

One thing you might want to do is set a base path for the API so that its endpoints are distinct and don't collide with any controllers you write. To adjust the base path for the API, set the `spring.data.rest.base-path` property as shown next:

```
spring:
  data:
    rest:
      base-path: /data-api
```

This sets the base path for Spring Data REST endpoints to /data-api. Although you can set the base path to anything you'd like, the choice of /data-api ensures that endpoints exposed by Spring Data REST don't collide with any other controllers, including those whose path begins with "/api" that we created earlier in this chapter. Consequently, the ingredients endpoint is now /data-api/ingredients. Now give this new base path a spin by requesting a list of tacos as follows:

```
$ curl http://localhost:8080/data-api/tacos
{
  "timestamp": "2018-02-11T16:22:12.381+0000",
  "status": 404,
  "error": "Not Found",
  "message": "No message available",
  "path": "/api/tacos"
}
```

Oh dear! That didn't work quite as expected. You have an `Ingredient` entity and an `IngredientRepository` interface, which Spring Data REST exposed with a /data-api/ingredients endpoint. So if you have a `Taco` entity and a `TacoRepository` interface, why doesn't Spring Data REST give you a /data-api/tacos endpoint?

7.2.1 Adjusting resource paths and relation names

Actually, Spring Data REST does give you an endpoint for working with tacos. But as clever as Spring Data REST can be, it shows itself to be a tiny bit less awesome in how it exposes the tacos endpoint.

When creating endpoints for Spring Data repositories, Spring Data REST tries to pluralize the associated entity class. For the `Ingredient` entity, the endpoint is /data-api/ingredients. For the `TacoOrder` entity, it's /data-api/orders. So far, so good.

But sometimes, such as with "taco," it trips up on a word and the pluralized version isn't quite right. As it turns out, Spring Data REST pluralized "taco" as "tacoes," so to make a request for tacos, you must play along and request /data-api/tacoes, as shown here:

```
$ curl localhost:8080/data-api/tacoes
{
  "_embedded" : {
    "tacoes" : [ {
      "name" : "Carnivore",
      "createdAt" : "2018-02-11T17:01:32.999+0000",
      "_links" : {
        "self" : {
          "href" : "http://localhost:8080/data-api/tacoes/2"
        },
```

```
        "taco" : {
          "href" : "http://localhost:8080/data-api/tacoes/2"
        },
        "ingredients" : {
          "href" : "http://localhost:8080/data-api/tacoes/2/ingredients"
        }
      }
    }]
  },
  "page" : {
    "size" : 20,
    "totalElements" : 3,
    "totalPages" : 1,
    "number" : 0
  }
}
```

You may be wondering how I knew that "taco" would be mispluralized as "tacoes." As it turns out, Spring Data REST also exposes a home resource that lists links for all exposed endpoints. Just make a `GET` request to the API base path to get the goods as follows:

```
$ curl localhost:8080/api
{
  "_links" : {
    "orders" : {
      "href" : "http://localhost:8080/data-api/orders"
    },
    "ingredients" : {
      "href" : "http://localhost:8080/data-api/ingredients"
    },
    "tacoes" : {
      "href" : "http://localhost:8080/data-api/tacoes{?page,size,sort}",
      "templated" : true
    },
    "users" : {
      "href" : "http://localhost:8080/data-api/users"
    },
    "profile" : {
      "href" : "http://localhost:8080/data-api/profile"
    }
  }
}
```

As you can see, the home resource shows the links for all of your entities. Everything looks good, except for the `tacoes` link, where both the relation name and the URL have that odd pluralization of "taco."

The good news is that you don't have to accept this little quirk of Spring Data REST. By adding the following simple annotation to the `Taco` class, you can tweak both the relation name and that path:

```
@Data
@Entity
```

```
@RestResource(rel="tacos", path="tacos")
public class Taco {
  ...
}
```

The `@RestResource` annotation lets you give the entity any relation name and path you want. In this case, you're setting them both to `"tacos"`. Now when you request the home resource, you see the `tacos` link with correct pluralization, as shown next:

```
"tacos" : {
  "href" : "http://localhost:8080/data-api/tacos{?page,size,sort}",
  "templated" : true
},
```

This also sorts out the path for the endpoint so that you can issue requests against /data-api/tacos to work with taco resources.

Speaking of sorting things out, let's look at how you can sort the results from Spring Data REST endpoints.

7.2.2 Paging and sorting

You may have noticed that the links in the home resource all offer optional `page`, `size`, and `sort` parameters. By default, requests to a collection resource such as /data-api/tacos will return up to 20 items per page from the first page. But you can adjust the page size and the page displayed by specifying the `page` and `size` parameters in your request.

For example, to request the first page of tacos where the page size is `5`, you can issue the following `GET` request (using `curl`):

```
$ curl "localhost:8080/data-api/tacos?size=5"
```

Assuming there are more than five tacos to be seen, you can request the second page of tacos by adding the `page` parameter as follows:

```
$ curl "localhost:8080/data-api/tacos?size=5&page=1"
```

Notice that the `page` parameter is zero-based, which means that asking for page `1` is actually asking for the second page. (You'll also note that many command-line shells trip up over the ampersand in the request, which is why I quoted the whole URL in the preceding `curl` command.)

The `sort` parameter lets you sort the resulting list by any property of the entity. For example, you need a way to fetch the 12 most recently created tacos for the UI to display. You can do that by specifying the following mix of paging and sorting parameters:

```
$ curl "localhost:8080/data-api/tacos?sort=createdAt,desc&page=0&size=12"
```

Here the `sort` parameter specifies that you should sort by the `createdDate` property and that it should be sorted in descending order (so that the newest tacos are first). The `page` and `size` parameters specify that you should see the first page of 12 tacos.

This is precisely what the UI needs to show the most recently created tacos. It's approximately the same as the /api/tacos?recent endpoint you defined in `TacoController` earlier in this chapter.

Now let's switch gears and see how to write client code to consume the API endpoints we've created.

7.3 Consuming REST services

Have you ever gone to a movie and, as the movie starts, discovered that you were the only person in the theater? It certainly is a wonderful experience to have what is essentially a private viewing of a movie. You can pick whatever seat you want, talk back to the characters onscreen, and maybe even open your phone and tweet about it without anyone getting angry for disrupting their movie-watching experience. And the best part is that nobody else is there ruining the movie for you, either!

This hasn't happened to me often. But when it has, I have wondered what would have happened if I hadn't shown up. Would they still have shown the film? Would the hero still have saved the day? Would the theater staff still have cleaned the theater after the movie was over?

A movie without an audience is kind of like an API without a client. It's ready to accept and provide data, but if the API is never invoked, is it really an API? Like Schrödinger's cat, we can't know if the API is active or returning HTTP 404 responses until we issue a request to it.

It's not uncommon for Spring applications to both provide an API and make requests to another application's API. In fact, this is becoming prevalent in the world of microservices. Therefore, it's worthwhile to spend a moment looking at how to use Spring to interact with REST APIs.

A Spring application can consume a REST API with the following:

- *`RestTemplate`*—A straightforward, synchronous REST client provided by the core Spring Framework.
- *Traverson*—A wrapper around Spring's `RestTemplate`, provided by Spring HATEOAS, to enable a hyperlink-aware, synchronous REST client. Inspired from a JavaScript library of the same name.
- *`WebClient`*—A reactive, asynchronous REST client.

For now, we'll focus on creating clients with `RestTemplate`. I'll defer discussion of `WebClient` until we cover Spring's reactive web framework in chapter 12. And if you're interested in writing hyperlink-aware clients, check out the Traverson documentation at http://mng.bz/aZno.

There's a lot that goes into interacting with a REST resource from the client's perspective—mostly tedium and boilerplate. Working with low-level HTTP libraries, the client needs to create a client instance and a request object, execute the request, interpret the response, map the response to domain objects, and handle any exceptions that may be thrown along the way. And all of this boilerplate is repeated, regardless of what HTTP request is sent.

To avoid such boilerplate code, Spring provides `RestTemplate`. Just as `JdbcTemplate` handles the ugly parts of working with JDBC, `RestTemplate` frees you from dealing with the tedium of consuming REST resources.

`RestTemplate` provides 41 methods for interacting with REST resources. Rather than examine all of the methods that it offers, it's easier to consider only a dozen unique operations, each overloaded to equal the complete set of 41 methods. The 12 operations are described in table 7.2.

Table 7.2 `RestTemplate` defines 12 unique operations, each of which is overloaded, providing a total of 41 methods.

Method	Description
`delete(…)`	Performs an HTTP `DELETE` request on a resource at a specified URL
`exchange(…)`	Executes a specified HTTP method against a URL, returning a `ResponseEntity` containing an object mapped from the response body
`execute(…)`	Executes a specified HTTP method against a URL, returning an object mapped from the response body
`getForEntity(…)`	Sends an HTTP `GET` request, returning a `ResponseEntity` containing an object mapped from the response body
`getForObject(…)`	Sends an HTTP `GET` request, returning an object mapped from a response body
`headForHeaders(…)`	Sends an HTTP `HEAD` request, returning the HTTP headers for the specified resource URL
`optionsForAllow(…)`	Sends an HTTP `OPTIONS` request, returning the `Allow` header for the specified URL
`patchForObject(…)`	Sends an HTTP `PATCH` request, returning the resulting object mapped from the response body
`postForEntity(…)`	`POST`s data to a URL, returning a `ResponseEntity` containing an object mapped from the response body
`postForLocation(…)`	`POST`s data to a URL, returning the URL of the newly created resource
`postForObject(…)`	`POST`s data to a URL, returning an object mapped from the response body
`put(…)`	`PUT`s resource data to the specified URL

With the exception of `TRACE`, `RestTemplate` has at least one method for each of the standard HTTP methods. In addition, `execute()` and `exchange()` provide lower-level, general-purpose methods for sending requests with any HTTP method.

Most of the methods in table 7.2 are overloaded into the following three method forms:

- One accepts a `String` URL specification with URL parameters specified in a variable argument list.

- One accepts a `String` URL specification with URL parameters specified in a `Map<String,String>`.
- One accepts a `java.net.URI` as the URL specification, with no support for parameterized URLs.

Once you get to know the 12 operations provided by `RestTemplate` and how each of the variant forms works, you'll be well on your way to writing resource-consuming REST clients.

To use `RestTemplate`, you'll either need to create an instance at the point you need it, as follows:

```
RestTemplate rest = new RestTemplate();
```

or you can declare it as a bean and inject it where you need it, as shown next:

```
@Bean
public RestTemplate restTemplate() {
  return new RestTemplate();
}
```

Let's survey `RestTemplate`'s operations by looking at those that support the four primary HTTP methods: `GET`, `PUT`, `DELETE`, and `POST`. We'll start with `getForObject()` and `getForEntity()`—the `GET` methods.

7.3.1 GETting resources

Suppose that you want to fetch an ingredient from the Taco Cloud API. For that, you can use `RestTemplate`'s `getForObject()` to fetch the ingredient. For example, the following code uses `RestTemplate` to fetch an `Ingredient` object by its ID:

```
public Ingredient getIngredientById(String ingredientId) {
  return rest.getForObject("http://localhost:8080/ingredients/{id}",
                           Ingredient.class, ingredientId);
}
```

Here you're using the `getForObject()` variant that accepts a `String` URL and uses a variable list for URL variables. The `ingredientId` parameter passed into `getForObject()` is used to fill in the `{id}` placeholder in the given URL. Although there's only one URL variable in this example, it's important to know that the variable parameters are assigned to the placeholders in the order that they're given.

The second parameter to `getForObject()` is the type that the response should be bound to. In this case, the response data (that's likely in JSON format) should be deserialized into an `Ingredient` object that will be returned.

Alternatively, you can use a `Map` to specify the URL variables, as shown next:

```
public Ingredient getIngredientById(String ingredientId) {
  Map<String, String> urlVariables = new HashMap<>();
  urlVariables.put("id", ingredientId);
```

```
  return rest.getForObject("http://localhost:8080/ingredients/{id}",
      Ingredient.class, urlVariables);
}
```

In this case, the value of `ingredientId` is mapped to a key of `id`. When the request is made, the `{id}` placeholder is replaced by the map entry whose key is `id`.

Using a `URI` parameter is a bit more involved, requiring that you construct a `URI` object before calling `getForObject()`. Otherwise, it's similar to both of the other variants, as shown here:

```
public Ingredient getIngredientById(String ingredientId) {
  Map<String, String> urlVariables = new HashMap<>();
  urlVariables.put("id", ingredientId);
  URI url = UriComponentsBuilder
            .fromHttpUrl("http://localhost:8080/ingredients/{id}")
            .build(urlVariables);
  return rest.getForObject(url, Ingredient.class);
}
```

Here the `URI` object is defined from a `String` specification, and its placeholders filled in from entries in a `Map`, much like the previous variant of `getForObject()`. The `getForObject()` method is a no-nonsense way of fetching a resource. But if the client needs more than the payload body, you may want to consider using `getForEntity()`.

`getForEntity()` works in much the same way as `getForObject()`, but instead of returning a domain object that represents the response's payload, it returns a `ResponseEntity` object that wraps that domain object. The `ResponseEntity` gives access to additional response details, such as the response headers.

For example, suppose that in addition to the ingredient data, you want to inspect the `Date` header from the response. With `getForEntity()` that becomes straightforward, as shown in the following code:

```
public Ingredient getIngredientById(String ingredientId) {
  ResponseEntity<Ingredient> responseEntity =
      rest.getForEntity("http://localhost:8080/ingredients/{id}",
          Ingredient.class, ingredientId);
  log.info("Fetched time: {}",
          responseEntity.getHeaders().getDate());
  return responseEntity.getBody();
}
```

The `getForEntity()` method is overloaded with the same parameters as `getForObject()`, so you can provide the URL variables as a variable list parameter or call `getForEntity()` with a `URI` object.

7.3.2 PUTting resources

For sending HTTP PUT requests, `RestTemplate` offers the `put()` method. All three overloaded variants of `put()` accept an `Object` that is to be serialized and sent to the given URL. As for the URL itself, it can be specified as a `URI` object or as a `String`. And

like `getForObject()` and `getForEntity()`, the URL variables can be provided as either a variable argument list or as a `Map`.

Suppose that you want to replace an ingredient resource with the data from a new `Ingredient` object. The following code should do the trick:

```
public void updateIngredient(Ingredient ingredient) {
  rest.put("http://localhost:8080/ingredients/{id}",
        ingredient, ingredient.getId());
}
```

Here the URL is given as a `String` and has a placeholder that's substituted by the given `Ingredient` object's `id` property. The data to be sent is the `Ingredient` object itself. The `put()` method returns `void`, so there's nothing you need to do to handle a return value.

7.3.3 DELETEing resources

Suppose that Taco Cloud no longer offers an ingredient and wants it completely removed as an option. To make that happen, you can call the `delete()` method from `RestTemplate` as follows:

```
public void deleteIngredient(Ingredient ingredient) {
  rest.delete("http://localhost:8080/ingredients/{id}",
      ingredient.getId());
}
```

In this example, only the URL (specified as a `String`) and a URL variable value are given to `delete()`. But as with the other `RestTemplate` methods, the URL could be specified as a `URI` object or the URL parameters given as a `Map`.

7.3.4 POSTing resource data

Now let's say that you add a new ingredient to the Taco Cloud menu. An HTTP `POST` request to the …/ingredients endpoint with ingredient data in the request body will make that happen. `RestTemplate` has three ways of sending a `POST` request, each of which has the same overloaded variants for specifying the URL. If you wanted to receive the newly created `Ingredient` resource after the `POST` request, you'd use `postForObject()` like this:

```
public Ingredient createIngredient(Ingredient ingredient) {
  return rest.postForObject("http://localhost:8080/ingredients",
      ingredient, Ingredient.class);
}
```

This variant of the `postForObject()` method takes a `String` URL specification, the object to be posted to the server, and the domain type that the response body should be bound to. Although you aren't taking advantage of it in this case, a fourth parameter could be a `Map` of the URL variable value or a variable list of parameters to substitute into the URL.

If your client has more need for the location of the newly created resource, then you can call `postForLocation()` instead, as shown here:

```
public java.net.URI createIngredient(Ingredient ingredient) {
  return rest.postForLocation("http://localhost:8080/ingredients",
      ingredient);
}
```

Notice that `postForLocation()` works much like `postForObject()`, with the exception that it returns a `URI` of the newly created resource instead of the resource object itself. The `URI` returned is derived from the response's `Location` header. In the off chance that you need both the location and response payload, you can call `postForEntity()` like so:

```
public Ingredient createIngredient(Ingredient ingredient) {
  ResponseEntity<Ingredient> responseEntity =
        rest.postForEntity("http://localhost:8080/ingredients",
                           ingredient,
                           Ingredient.class);
  log.info("New resource created at {}",
           responseEntity.getHeaders().getLocation());
  return responseEntity.getBody();
}
```

Although the methods of `RestTemplate` differ in their purpose, they're quite similar in how they're used. This makes it easy to become proficient with `RestTemplate` and use it in your client code.

Summary

- REST endpoints can be created with Spring MVC, with controllers that follow the same programming model as browser-targeted controllers.
- Controller handler methods can either be annotated with `@ResponseBody` or return `ResponseEntity` objects to bypass the model and view and write data directly to the response body.
- The `@RestController` annotation simplifies REST controllers, eliminating the need to use `@ResponseBody` on handler methods.
- Spring Data repositories can automatically be exposed as REST APIs using Spring Data REST.

8 Securing REST

This chapter covers

- Securing APIs with OAuth 2
- Creating an authorization server
- Adding a resource server to an API
- Consuming OAuth 2–secured APIs

Have you ever taken advantage of valet parking? It's a simple concept: you hand your car keys to a valet near the entrance of a store, hotel, theater, or restaurant, and they deal with the hassle of finding a parking space for you. And then they return your car to you when you ask for it. Maybe it's because I've seen *Ferris Bueller's Day Off* too many times, but I'm always reluctant to hand my car keys to a stranger and hope that they take good care of my vehicle for me.

Nonetheless, valet parking involves granting trust to someone to take care of your car. Many newer cars provide a "valet key," a special key that can be used only to open the car doors and start the engine. This way the amount of trust that you are granting is limited in scope. The valet cannot open the glove compartment or the trunk with the valet key.

In a distributed application, trust is critical between software systems. Even in a simple situation where a client application consumes a backend API, it's important

that the client is trusted and anyone else attempting to use that same API is blocked out. And, like the valet, the amount of trust you grant to a client should be limited to only the functions necessary for the client to do its job.

Securing a REST API is different from securing a browser-based web application. In this chapter, we're going to look at OAuth 2, an authorization specification created specifically for API security. In doing so, we'll look at Spring Security's support for OAuth 2. But first, let's set the stage by seeing how OAuth 2 works.

8.1 Introducing OAuth 2

Suppose that we want to create a new back-office application for managing the Taco Cloud application. More specifically, let's say that we want this new application to be able to manage the ingredients available on the main Taco Cloud website.

Before we start writing code for the administrative application, we'll need to add a handful of new endpoints to the Taco Cloud API to support ingredient management. The REST controller in the following listing offers three endpoints for listing, adding, and deleting ingredients.

Listing 8.1 A controller to manage available ingredients

```
package tacos.web.api;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.CrossOrigin;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.ResponseStatus;
import org.springframework.web.bind.annotation.RestController;

import tacos.Ingredient;
import tacos.data.IngredientRepository;

@RestController
@RequestMapping(path="/api/ingredients", produces="application/json")
@CrossOrigin(origins="http://localhost:8080")
public class IngredientController {

  private IngredientRepository repo;

  @Autowired
  public IngredientController(IngredientRepository repo) {
    this.repo = repo;
  }

  @GetMapping
  public Iterable<Ingredient> allIngredients() {
```

```
    return repo.findAll();
  }

  @PostMapping
  @ResponseStatus(HttpStatus.CREATED)
  public Ingredient saveIngredient(@RequestBody Ingredient ingredient) {
    return repo.save(ingredient);
  }

  @DeleteMapping("/{id}")
  @ResponseStatus(HttpStatus.NO_CONTENT)
  public void deleteIngredient(@PathVariable("id") String ingredientId) {
    repo.deleteById(ingredientId);
  }

}
```

Great! Now all we need to do is get started on the administrative application, calling those endpoints on the main Taco Cloud application as needed to add and delete ingredients.

But wait—there's no security around that API yet. If our backend application can make HTTP requests to add and delete ingredients, so can anyone else. Even using the `curl` command-line client, someone could add a new ingredient like this:

```
$ curl localhost:8080/ingredients \
  -H"Content-type: application/json" \
  -d'{"id":"FISH","name":"Stinky Fish", "type":"PROTEIN"}'
```

They could even use `curl` to delete existing ingredients[1] as follows:

```
$ curl localhost:8080/ingredients/GRBF -X DELETE
```

This API is part of the main application and available to the world; in fact, the `GET` endpoint is used by the user interface of the main application in home.html. Therefore, it's clear that we'll need to secure at least the `POST` and `DELETE` endpoints.

One option is to use HTTP Basic authentication to secure the /ingredients endpoints. This could be done by adding `@PreAuthorize` to the handler methods like this:

```
@PostMapping
@PreAuthorize("#{hasRole('ADMIN')}")
public Ingredient saveIngredient(@RequestBody Ingredient ingredient) {
  return repo.save(ingredient);
}

@DeleteMapping("/{id}")
@PreAuthorize("#{hasRole('ADMIN')}")
public void deleteIngredient(@PathVariable("id") String ingredientId) {
```

[1] Depending on the database and schema in play, integrity constraints may prevent a deletion from happening if an ingredient is already part of an existing taco. But it still may be possible to delete an ingredient if the database schema allows it.

```
  repo.deleteById(ingredientId);
}
```

Or, the endpoints could be secured in the security configuration like this:

```
@Override
protected void configure(HttpSecurity http) throws Exception {
  http
    .authorizeRequests()
      .antMatchers(HttpMethod.POST, "/ingredients").hasRole("ADMIN")
      .antMatchers(HttpMethod.DELETE, "/ingredients/**").hasRole("ADMIN")

      ...
}
```

Whether or not to use the "ROLE_" prefix

Authorities in Spring Security can take several forms, including roles, permissions, and (as we'll see later) OAuth2 scopes. Roles, specifically, are a specialized form of authority that are prefixed with `"ROLE_"`.

When working with methods or SpEL expressions that deal directly with roles, such as `hasRole()`, the `"ROLE_"` prefix is inferred. Thus, a call to `hasRole("ADMIN")` is internally checking for an authority whose name is `"ROLE_ADMIN"`. You do not need to explicitly use the `"ROLE_"` prefix when calling these methods and functions (and, in fact, doing so will result in a double `"ROLE_"` prefix).

Other Spring Security methods and functions that deal with authority more generically can also be used to check for roles. But in those cases, you must explicitly add the `"ROLE_"` prefix. For example, if you chose to use `hasAuthority()` instead of `hasRole()`, you'd need to pass in `"ROLE_ADMIN"` instead of `"ADMIN"`.

Either way, the ability to submit `POST` or `DELETE` requests to /ingredients will require that the submitter also provide credentials that have `"ROLE_ADMIN"` authority. For example, using `curl`, the credentials can be specified with the `-u` parameter, as shown here:

```
$ curl localhost:8080/ingredients \
  -H"Content-type: application/json" \
  -d'{"id":"FISH","name":"Stinky Fish", "type":"PROTEIN"}' \
  -u admin:l3tm31n
```

Although HTTP Basic will lock down the API, it is rather . . . um . . . basic. It requires that the client and the API share common knowledge of the user credentials, possibly duplicating information. Moreover, although HTTP Basic credentials are Base64-encoded in the header of the request, if a hacker were to somehow intercept the request, the credentials could easily be obtained, decoded, and used for evil purposes. If that were to happen, the password would need to be changed, thus requiring an update and reauthentication in all clients.

What if instead of requiring that the admin user identify themselves on every request, the API just asks for some token that proves that they are authorized to access the resources? This would be roughly like a ticket to a sporting event. To enter the game, the person at the turnstiles doesn't need to know who you are; they just need to know that you have a valid ticket. If so, then you are allowed access.

That's roughly how OAuth 2 authorization works. Clients request an access token—analogous to a valet key—from an authorization server, with the express permission of a user. That token allows them to interact with an API on behalf of the user who authorized the client. At any point, the token could expire or be revoked, without requiring that the user's password be changed. In such cases, the client would just need to request a new access token to be able to continue acting on the user's behalf. This flow is illustrated in figure 8.1.

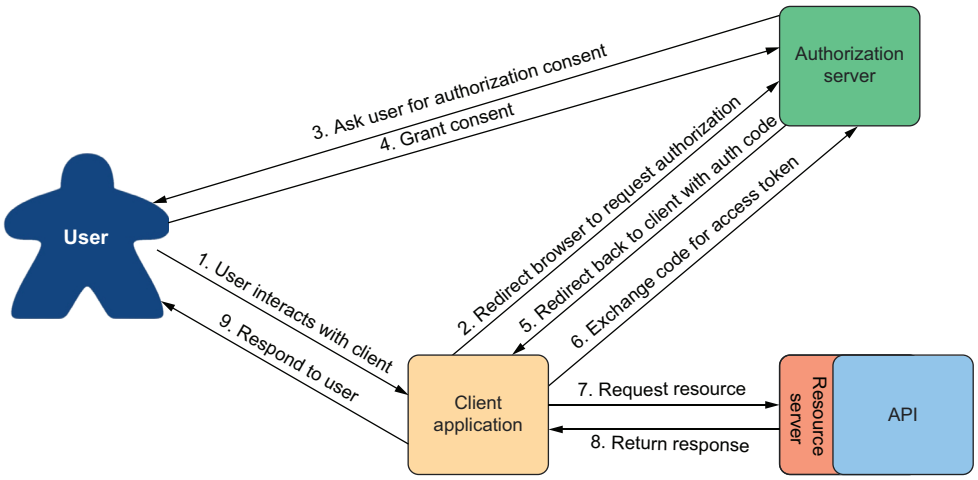

Figure 8.1 The OAuth 2 authorization code flow

OAuth 2 is a very rich security specification that offers a lot of ways to use it. The flow described in figure 8.1 is called *authorization code grant*. Other flows supported by the OAuth 2 specification include these:

- *Implicit grant*—Like authorization code grant, implicit grant redirects the user's browser to the authorization server to get user consent. But when redirecting back, rather than provide an authorization code in the request, the access token is granted implicitly in the request. Although originally designed for JavaScript clients running in a browser, this flow is not generally recommended anymore, and authorization code grant is preferred.
- *User credentials (or password) grant*—In this flow, no redirect takes place, and there may not even be a web browser involved. Instead, the client application obtains the

user's credentials and exchanges them directly for an access token. This flow seems suitable for clients that are not browser based, but modern applications often favor asking the user to go to a website in their browser and perform authorization code grant to avoid having to handle the user's credentials.

- *Client credentials grant*—This flow is like user credentials grant, except that instead of exchanging a user's credentials for an access token, the client exchanges its own credentials for an access token. However, the token granted is limited in scope to performing non-user-focused operations and can't be used to act on behalf of a user.

For our purposes, we're going to focus on the authorization code grant flow to obtain a JSON Web Token (JWT) access token. This will involve creating a handful of applications that work together, including the following:

- *The authorization server*—An authorization server's job is to obtain permission from a user on behalf of a client application. If the user grants permission, then the authorization server gives an access token to the client application that it can use to gain authenticated access to an API.
- *The resource server*—A resource server is just another name for an API that is secured by OAuth 2. Although the resource server is part of the API itself, for the sake of discussion, the two are often treated as two distinct concepts. The resource server restricts access to its resources unless the request provides a valid access token with the necessary permission scope. For our purposes, the Taco Cloud API we started in chapter 7 will serve as our resource server, once we add a bit of security configuration to it.
- *The client application*—The client application is an application that wants to consume an API but needs permission to do so. We'll build a simple administrative application for Taco Cloud to be able to add new ingredients.
- *The user*—This is the human who uses the client application and grants the application permission to access the resource server's API on their behalf.

In the authorization code grant flow, a series of browser redirects between the client application and the authorization server occurs as the client obtains an access token. It starts with the client redirecting the user's browser to the authorization server, asking for specific permissions (or "scope"). The authorization server then asks the user to log in and consent to the requested permissions. After the user has granted consent, the authorization server redirects the browser back to the client with a code that the client can then exchange for an access token. Once the client has the access token, it can then be used to interact with the resource server API by passing it in the `"Authorization"` header of every request.

Although we're going to restrict our focus on a specific use of OAuth 2, you are encouraged to dig deeper into the subject by reading the OAuth 2 specification (https://oauth.net/2/) or reading any one of the following books on the subject:

- *OAuth 2 in Action*: https://www.manning.com/books/oauth-2-in-action
- *Microservices Security in Action*: https://www.manning.com/books/microservices-security-in-action
- *API Security in Action*: https://www.manning.com/books/api-security-in-action

You might also want to have a look at a liveProject called "Protecting User Data with Spring Security and OAuth2" (http://mng.bz/4KdD).

For several years, a project called Spring Security for OAuth provided support for both OAuth 1.0a and OAuth 2. It was separate from Spring Security but developed by the same team. In recent years, however, the Spring Security team has absorbed the client and resource server components into Spring Security itself.

As for the authorization server, it was decided that it not be included in Spring Security. Instead, developers are encouraged to use authorization servers from various vendors such as Okta, Google, and others. But, due to demand from the developer community, the Spring Security team started a Spring Authorization Server project.[2] This project is labeled as "experimental" and is intended to eventually be community driven, but it serves as a great way to get started with OAuth 2 without signing up for one of those other authorization server implementations.

Throughout the rest of this chapter, we're going to see how to use OAuth 2 using Spring Security. Along the way, we'll create two new projects, an authorization server project and a client project, and we'll modify our existing Taco Cloud project such that its API acts as a resource server. We'll start by creating an authorization server using the Spring Authorization Server project.

8.2 Creating an authorization server

An authorization server's job is primarily to issue an access token on behalf of a user. As mentioned earlier, we have several authorization server implementations to choose from, but we're going to use Spring Authorization Server for our project. Spring Authorization Server is experimental and doesn't implement all of the OAuth 2 grant types, but it does implement the authorization code grant and client credentials grant.

The authorization server is a distinct application from any application that provides the API and is also distinct from the client. Therefore, to get started with Spring Authorization Server, you'll want to create a new Spring Boot project, choosing (at least) the web and security starters. For our authorization server, users will be stored in a relational database using JPA, so be sure to add the JPA starter and H2 dependencies as well. And, if you're using Lombok to handle getters, setters, constructors, and whatnot, then be sure to include it as well.

Spring Authorization Server isn't (yet) available as a dependency from the Initializr. So once your project has been created, you'll need to manually add the Spring

[2] See http://mng.bz/QqGR.

Authorization Server dependency to your build. For example, here's the Maven dependency you'll need to include in your pom.xml file:

```
<dependency>
    <groupId>org.springframework.security.experimental</groupId>
    <artifactId>spring-security-oauth2-authorization-server</artifactId>
    <version>0.1.2</version>
</dependency>
```

Next, because we'll be running this all on our development machines (at least for now), you'll want to make sure that there's not a port conflict between the main Taco Cloud application and the authorization server. Adding the following entry to the project's application.yml file will make the authorization server available on port 9000:

```
server:
  port: 9000
```

Now let's dive into the essential security configuration that will be used by the authorization server. The next code listing shows a very simple Spring Security configuration class that enables form-based login and requires that all requests be authenticated.

Listing 8.2 Essential security configuration for form-based login

```
package tacos.authorization;
import org.springframework.context.annotation.Bean;
import org.springframework.security.config.annotation.web.builders.
            HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.
            EnableWebSecurity;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;

import tacos.authorization.users.UserRepository;

@EnableWebSecurity
public class SecurityConfig {

    @Bean
    SecurityFilterChain defaultSecurityFilterChain(HttpSecurity http)
            throws Exception {
        return http
            .authorizeRequests(authorizeRequests ->
                authorizeRequests.anyRequest().authenticated()
            )

            .formLogin()

            .and().build();
    }
```

```
    @Bean
    UserDetailsService userDetailsService(UserRepository userRepo) {
      return username -> userRepo.findByUsername(username);
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
      return new BCryptPasswordEncoder();
    }
}
```

Notice that the `UserDetailsService` bean works with a `TacoUserRepository` to look up users by their username. To get on with configuring the authorization server itself, we'll skip over the specifics of `TacoUserRepository`, but suffice it to say that it looks a lot like some of the Spring Data–based repositories we've created since chapter 3.

The only thing worth noting about the `TacoUserRepository` is that (for convenience in testing) you could use it in a `CommandLineRunner` bean to prepopulate the database with a couple of test users as follows:

```
@Bean
public ApplicationRunner dataLoader(
        UserRepository repo, PasswordEncoder encoder) {
  return args -> {
    repo.save(
        new User("habuma", encoder.encode("password"), "ROLE_ADMIN"));
    repo.save(
        new User("tacochef", encoder.encode("password"), "ROLE_ADMIN"));
  };
}
```

Now we can start applying configuration to enable an authorization server. The first step in configuring an authorization server is to create a new configuration class that imports some common configuration for an authorization server. The following code for `AuthorizationServerConfig` is a good start:

```
@Configuration(proxyBeanMethods = false)
public class AuthorizationServerConfig {

  @Bean
  @Order(Ordered.HIGHEST_PRECEDENCE)
  public SecurityFilterChain
     authorizationServerSecurityFilterChain(HttpSecurity http) throws
     Exception {
    OAuth2AuthorizationServerConfiguration
        .applyDefaultSecurity(http);
    return http
        .formLogin(Customizer.withDefaults())
        .build();
  }
```

```
    ...
}
```

The `authorizationServerSecurityFilterChain()` bean method defines a `SecurityFilterChain` that sets up some default behavior for the OAuth 2 authorization server and a default form login page. The `@Order` annotation is given `Ordered.HIGHEST_PRECEDENCE` to ensure that if for some reason there are other beans of this type declared, this one takes precedence over the others.

For the most part, this is a boilerplate configuration. If you want, feel free to dive in a little deeper and customize the configuration. For now, we're just going to go with the defaults.

One component that isn't boilerplate, and thus not provided by `OAuth2AuthorizationServerConfiguration`, is the client repository. A client repository is analogous to a user details service or user repository, except that instead of maintaining details about users, it maintains details about clients that might be asking for authorization on behalf of users. It is defined by the `RegisteredClientRepository` interface, which looks like this:

```
public interface RegisteredClientRepository {

    @Nullable
    RegisteredClient findById(String id);

    @Nullable
    RegisteredClient findByClientId(String clientId);

}
```

In a production setting, you might write a custom implementations of `RegisteredClientRepository` to retrieve client details from a database or from some other source. But out of the box, Spring Authorization Server offers an in-memory implementation that is perfect for demonstration and testing purposes. You're encouraged to implement `RegisteredClientRepository` however you see fit. But for our purposes, we'll use the in-memory implementation to register a single client with the authorization server. Add the following bean method to `AuthorizationServerConfig`:

```
@Bean
public RegisteredClientRepository registeredClientRepository(
        PasswordEncoder passwordEncoder) {
  RegisteredClient registeredClient =
    RegisteredClient.withId(UUID.randomUUID().toString())
      .clientId("taco-admin-client")
      .clientSecret(passwordEncoder.encode("secret"))
      .clientAuthenticationMethod(
              ClientAuthenticationMethod.CLIENT_SECRET_BASIC)
      .authorizationGrantType(AuthorizationGrantType.AUTHORIZATION_CODE)
      .authorizationGrantType(AuthorizationGrantType.REFRESH_TOKEN)
```

```
        .redirectUri(
            "http://127.0.0.1:9090/login/oauth2/code/taco-admin-client")
        .scope("writeIngredients")
        .scope("deleteIngredients")
        .scope(OidcScopes.OPENID)
        .clientSettings(
            clientSettings -> clientSettings.requireUserConsent(true))
        .build();
    return new InMemoryRegisteredClientRepository(registeredClient);
}
```

As you can see, there are a lot of details that go into a `RegisteredClient`. But going from top to bottom, here's how our client is defined:

- *ID*—A random, unique identifier.
- *Client ID*—Analogous to a username, but instead of a user, it is a client. In this case, `"taco-admin-client"`.
- *Client secret*—Analogous to a password for the client. Here we're using the word `"secret"` for the client secret.
- *Authorization grant type*—The OAuth 2 grant types that this client will support. In this case, we're enabling authorization code and refresh token grants.
- *Redirect URL*—One or more registered URLs that the authorization server can redirect to after authorization has been granted. This adds another level of security, preventing some arbitrary application from receiving an authorization code that it could exchange for a token.
- *Scope*—One or more OAuth 2 scopes that this client is allowed to ask for. Here we are setting three scopes: `"writeIngredients"`, `"deleteIngredients"`, and the constant `OidcScopes.OPENID`, which resolves to `"openid"`. The `"openid"` scope will be necessary later when we use the authorization server as a single-sign-on solution for the Taco Cloud admin application.
- *Client settings*—This is a lambda that allows us to customize the client settings. In this case, we're requiring explicit user consent before granting the requested scope. Without this, the scope would be implicitly granted after the user logs in.

Finally, because our authorization server will be producing JWT tokens, the tokens will need to include a signature created using a JSON Web Key (JWK)[3] as the signing key. Therefore, we'll need a few beans to produce a JWK. Add the following bean method (and private helper methods) to `AuthorizationServerConfig` to handle that for us:

```
@Bean
  public JWKSource<SecurityContext> jwkSource()
          throws NoSuchAlgorithmException {
```

[3] See https://datatracker.ietf.org/doc/html/rfc7517.

```
  RSAKey rsaKey = generateRsa();
  JWKSet jwkSet = new JWKSet(rsaKey);
  return (jwkSelector, securityContext) -> jwkSelector.select(jwkSet);
}

private static RSAKey generateRsa() throws NoSuchAlgorithmException {
  KeyPair keyPair = generateRsaKey();
  RSAPublicKey publicKey = (RSAPublicKey) keyPair.getPublic();
  RSAPrivateKey privateKey = (RSAPrivateKey) keyPair.getPrivate();
  return new RSAKey.Builder(publicKey)
      .privateKey(privateKey)
      .keyID(UUID.randomUUID().toString())
      .build();
}

private static KeyPair generateRsaKey() throws NoSuchAlgorithmException {
    KeyPairGenerator keyPairGenerator =
   KeyPairGenerator.getInstance("RSA");
    keyPairGenerator.initialize(2048);
    return keyPairGenerator.generateKeyPair();
}

@Bean
public JwtDecoder jwtDecoder(JWKSource<SecurityContext> jwkSource) {
  return OAuth2AuthorizationServerConfiguration.jwtDecoder(jwkSource);
}
```

There appears to be a lot going on here. But to summarize, the `JWKSource` creates RSA 2048-bit key pairs that will be used to sign the token. The token will be signed using the private key. The resource server can then verify that the token received in a request is valid by obtaining the public key from the authorization server. We'll talk more about that when we create the resource server.

All of the pieces of our authorization server are now in place. All that's left to do is start it up and try it out. Build and run the application, and you should have an authorization server listening on port 9000.

Because we don't have a client yet, you can pretend to be a client using your web browser and the `curl` command-line tool. Start by pointing your web browser at http://localhost:9000/oauth2/authorize?response_type=code&client_id=tacoadmin-client&redirect_uri=http://127.0.0.1:9090/login/oauth2/code/taco-admin-client&-scope=writeIngredients+deleteIngredients.[4] You should see a login page that looks like figure 8.2.

[4] Notice that this and all URLs in this chapter are using "http://" URLs. This makes local development and testing easy. But in a production setting, you should always use "https://" URLs for increased security.

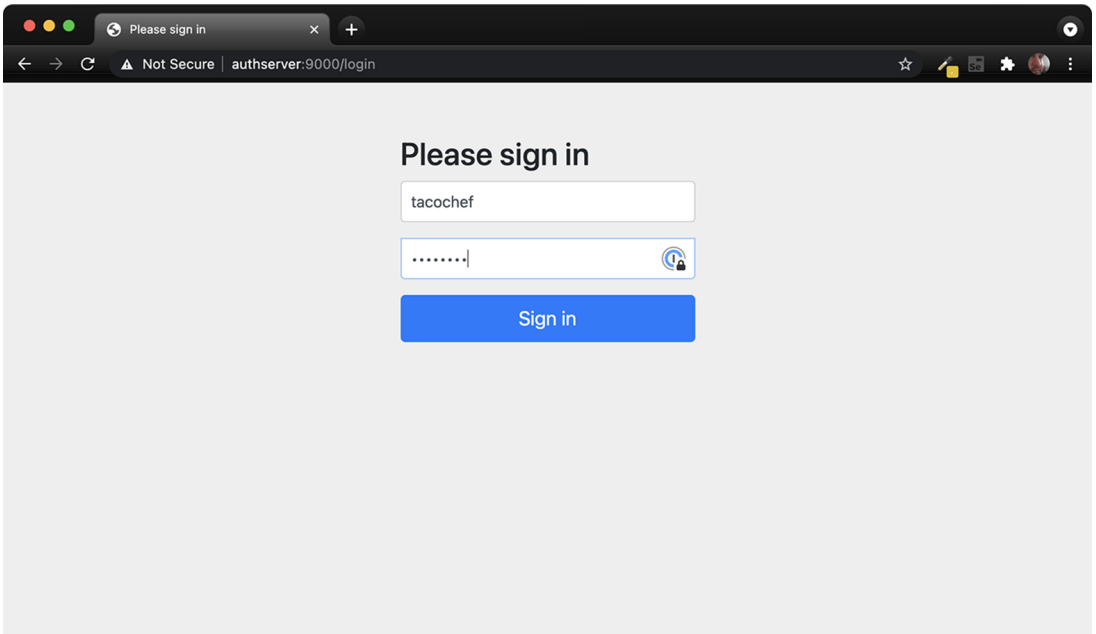

Figure 8.2 The authorization server login page

After logging in (with "tacochef" and "password," or some username-password combination in the database under `TacoUserRepository`), you'll be asked to consent to the requested scopes on a page that looks like figure 8.3.

After granting consent, the browser will be redirected back to the client URL. We don't have a client yet, so there's probably nothing there and you'll receive an error. But that's OK—we're pretending to be the client, so we'll obtain the authorization code from the URL ourselves.

Look in the browser's address bar, and you'll see that the URL has a `code` parameter. Copy the entire value of that parameter, and use it in the following `curl` command line in place of `$code`:

```
$ curl localhost:9000/oauth2/token \
    -H"Content-type: application/x-www-form-urlencoded" \
    -d"grant_type=authorization_code" \
    -d"redirect_uri=http://127.0.0.1:9090/login/oauth2/code/taco-admin-
     client" \
    -d"code=$code" \
    -u taco-admin-client:secret
```

Here we're exchanging the authorization code we received for an access token. The payload body is in "application/x-www-form-urlencoded" format and sends the grant type (`"authorization_code"`), the redirect URI (for additional security), and the

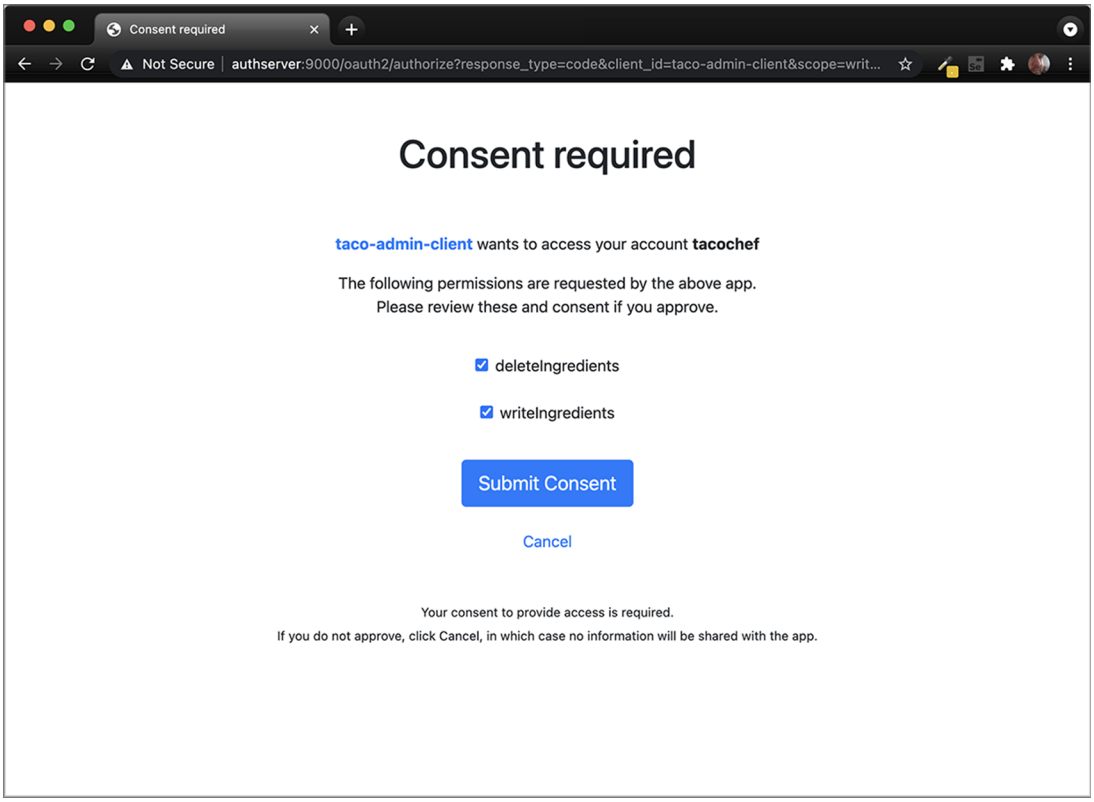

Figure 8.3 The authorization server consent page

authorization code itself. If all goes well, then you'll receive a JSON response that (when formatted) looks like this:

```
{
  "access_token":"eyJraWQ...",
  "refresh_token":"HOzHA5s...",
  "scope":"deleteIngredients writeIngredients",
  "token_type":"Bearer",
  "expires_in":"299"
}
```

The `"access_token"` property contains the access token that a client can use to make requests to the API. In reality, it is much longer than shown here. Likewise, the `"refresh_token"` has been abbreviated here to save space. But the access token can now be sent on requests to the resource server to gain access to resources requiring either the `"writeIngredients"` or `"deleteIngredients"` scope. The access token will expire in 299 seconds (or just less than 5 minutes), so we'll have to move quickly if

we're going to use it. But if it expires, then we can use the refresh token to obtain a new access token without going through the authorization flow all over again.

So, how can we use the access token? Presumably, we'll send it in a request to the Taco Cloud API as part of the `"Authorization"` header—perhaps something like this:

```
$ curl localhost:8080/ingredients \
  -H"Content-type: application/json" \
  -H"Authorization: Bearer eyJraWQ..." \
  -d'{"id":"FISH","name":"Stinky Fish", "type":"PROTEIN"}'
```

At this point, the token achieves nothing for us. That's because our Taco Cloud API hasn't been enabled to be a resource server yet. But in lieu of an actual resource server and client API, we can still inspect the access token by copying it and pasting into the form at https://jwt.io. The result will look something like figure 8.4.

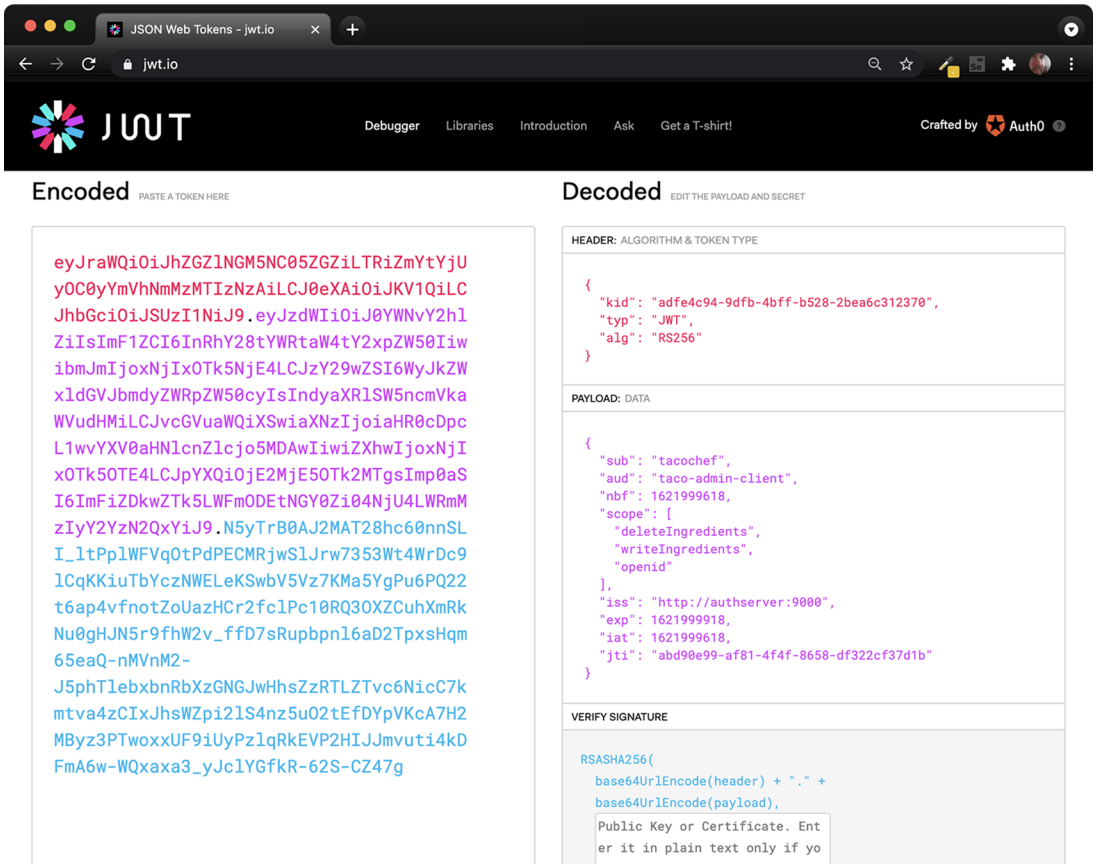

Figure 8.4 Decoding a JWT token at jwt.io

As you can see, the token is decoded into three parts: the header, the payload, and the signature. A closer look at the payload shows that this token was issued on behalf of the user named tacochef and the token has the `"writeIngredients"` and `"deleteIngredients"` scopes. Just what we asked for!

After about 5 minutes, the access token will expire. You can still inspect it in the debugger at https://jwti.io, but if it were given in a real request to an API, it would be rejected. But you can request a new access token without going through the authorization code grant flow again. All you need to do is make a new request to the authorization server using the `"refresh_token"` grant and passing the refresh token as the value of the `"refresh_token"` parameter. Using `curl`, such a request will look like this:

```
$ curl localhost:9000/oauth2/token \
    -H"Content-type: application/x-www-form-urlencoded" \
    -d"grant_type=refresh_token&refresh_token=HOzHA5s..." \
    -u taco-admin-client:secret
```

The response to this request will be the same as the response from the request that exchanged the authorization code for an access token initially, only with a fresh new access token.

Although it's fun to paste access tokens into https://jwt.io, the real power and purpose of the access token is to gain access to an API. So let's see how to enable a resource server on the Taco Cloud API.

8.3 Securing an API with a resource server

The resource server is actually just a filter that sits in front of an API, ensuring that requests for resources that require authorization include a valid access token with the required scope. Spring Security provides an OAuth2 resource server implementation that you can add to an existing API by adding the following dependency to the build for the project build as follows:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
</dependency>
```

You can also add the resource server dependency by selecting the "OAuth2 Resource Server" dependency from the Initializr when creating a project.

With the dependency in place, the next step is to declare that `POST` requests to /ingredients require the `"writeIngredients"` scope and that `DELETE` requests to /ingredients require the `"deleteIngredients"` scope. The following excerpt from the project's `SecurityConfig` class shows how to do that:

```
@Override
  protected void configure(HttpSecurity http) throws Exception {
    http
        .authorizeRequests()
       ...
```

```
        .antMatchers(HttpMethod.POST, "/api/ingredients")
            .hasAuthority("SCOPE_writeIngredients")
        .antMatchers(HttpMethod.DELETE, "/api//ingredients")
            .hasAuthority("SCOPE_deleteIngredients")
    ...
  }
```

For each of the endpoints, the `.hasAuthority()` method is called to specify the required scope. Notice that the scopes are prefixed with `"SCOPE_"` to indicate that they should be matched against OAuth 2 scopes in the access token given on the request to those resources.

In that same configuration class, we'll also need to enable the resource server, as shown next:

```
@Override
protected void configure(HttpSecurity http) throws Exception {
  http
      ...
      .and()
        .oauth2ResourceServer(oauth2 -> oauth2.jwt())
   ...
}
```

The `oauth2ResourceServer()` method here is given a lambda with which to configure the resource server. Here, it simply enables JWT tokens (as opposed to opaque tokens) so that the resource server can inspect the contents of the token to find what security claims it includes. Specifically, it will look to see that the token includes the `"write-Ingredients"` and/or `"deleteIngredients"` scope for the two endpoints we've secured.

It won't trust the token at face value, though. To be confident that the token was created by a trusted authorization server on behalf of a user, it will verify the token's signature using the public key that matches the private key that was used to create the token's signature. We'll need to configure the resource server to know where to obtain the public key, though. The following property will specify the JWK set URL on the authorization server from which the resource server will fetch the public key:

```
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          jwk-set-uri: http://localhost:9000/oauth2/jwks
```

And now our resource server is ready! Build the Taco Cloud application and start it up. Then you can try it out using `curl` like this:

```
$ curl localhost:8080/ingredients \
        -H"Content-type: application/json" \
        -d'{"id":"CRKT", "name":"Legless Crickets", "type":"PROTEIN"}'
```

The request should fail with an HTTP 401 response code. That's because we've configured the endpoint to require the `"writeIngredients"` scope for that endpoint, and we've not provided a valid access token with that scope on the request.

To make a successful request and add a new ingredient item, you'll need to obtain an access token using the flow we used in the previous section, making sure that we request the `"writeIngredients"` and `"deleteIngredients"` scopes when directing the browser to the authorization server. Then, provide the access token in the `"Authorization"` header using `curl` like this (substituting `"$token"` for the actual access token):

```
$ curl localhost:8080/ingredients \
    -H"Content-type: application/json" \
    -d'{"id":"SHMP", "name":"Coconut Shrimp", "type":"PROTEIN"}' \
    -H"Authorization: Bearer $token"
```

This time the new ingredient should be created. You can verify that by using `curl` or your chosen HTTP client to perform a `GET` request to the /ingredients endpoint as follows:

```
$ curl localhost:8080/ingredients
[
    {
        "id": "FLTO",
        "name": "Flour Tortilla",
        "type": "WRAP"
    },

    ...

    {
        "id": "SHMP",
        "name": "Coconut Shrimp",
        "type": "PROTEIN"
    }
]
```

Coconut Shrimp is now included at the end of the list of all of the ingredients returned from the /ingredients endpoint. Success!

Recall that the access token expires after 5 minutes. If you let the token expire, requests will start returning HTTP 401 responses again. But you can get a new access token by making a request to the authorization server using the refresh token that you got along with the access token (substituting the actual refresh token for `"$refreshToken"`), as shown here:

```
$ curl localhost:9000/oauth2/token \
    -H"Content-type: application/x-www-form-urlencoded" \
    -d"grant_type=refresh_token&refresh_token=$refreshToken" \
    -u taco-admin-client:secret
```

With a newly created access token, you can keep on creating new ingredients to your heart's content.

Now that we've secured the /ingredients endpoint, it's probably a good idea to apply the same techniques to secure other potentially sensitive endpoints in our API. The /orders endpoint, for example, should probably not be open for any kind of request, even HTTP `GET` requests, because it would allow a hacker to easily grab customer information. I'll leave it up to you to secure the /orders endpoint and the rest of the API as you see fit.

Administering the Taco Cloud application using `curl` works great for tinkering and getting to know how OAuth 2 tokens play a part in allowing access to a resource. But ultimately we want a real client application that can be used to manage ingredients. Let's now turn our attention to creating an OAuth-enabled client that will obtain access tokens and make requests to the API.

8.4 Developing the client

In the OAuth 2 authorization dance, the client application's role is to obtain an access token and make requests to the resource server on behalf of the user. Because we're using OAuth 2's authorization code flow, that means that when the client application determines that the user has not yet been authenticated, it should redirect the user's browser to the authorization server to get consent from the user. Then, when the authorization server redirects control back to the client, the client must exchange the authorization code it receives for the access token.

First things first: the client will need Spring Security's OAuth 2 client support in its classpath. The following starter dependency makes that happen:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-oauth2-client</artifactId>
</dependency>
```

Not only does this give the application OAuth 2 client capabilities that we'll exploit in a moment, but it also transitively brings in Spring Security itself. This enables us to write some security configuration for the application. The following `SecurityFilterChain` bean sets up Spring Security so that all requests require authentication:

```
@Bean
SecurityFilterChain defaultSecurityFilterChain(HttpSecurity http) throws
     Exception {
  http
    .authorizeRequests(
        authorizeRequests -> authorizeRequests.anyRequest().authenticated()
    )
    .oauth2Login(
      oauth2Login ->
      oauth2Login.loginPage("/oauth2/authorization/taco-admin-client"))
    .oauth2Client(withDefaults());
  return http.build();
}
```

What's more, this `SecurityFilterChain` bean also enables the client-side bits of OAuth 2. Specifically, it sets up a login page at the path /oauth2/authorization/taco-admin-client. But this is no ordinary login page that takes a username and password. Instead, it accepts an authorization code, exchanges it for an access token, and uses the access token to determine the identity of the user. Put another way, this is the path that the authorization server will redirect to after the user has granted permission.

We also need to configure details about the authorization server and our application's OAuth 2 client details. That is done in configuration properties, such as in the following application.yml file, which configures a client named `taco-admin-client`:

```
spring:
  security:
    oauth2:
      client:
        registration:
          taco-admin-client:
            provider: tacocloud
            client-id: taco-admin-client
            client-secret: secret
            authorization-grant-type: authorization_code
            redirect-uri:
                ➥ "http://127.0.0.1:9090/login/oauth2/code/{registrationId}"
            scope: writeIngredients,deleteIngredients,openid
```

This registers a client with the Spring Security OAuth 2 client named `taco-admin-client`. The registration details include the client's credentials (`client-id` and `client-secret`), the grant type (`authorization-grant-type`), the scopes being requested (`scope`), and the redirect URI (`redirect-uri`). Notice that the value given to `redirect-uri` has a placeholder that references the client's registration ID, which is `taco-admin-client`. Consequently, the redirect URI is set to http://127.0.0.1:9090/login/oauth2/code/taco-admin-client, which has the same path that we configured as the OAuth 2 login earlier.

But what about the authorization server itself? Where do we tell the client that it should redirect the user's browser? That's what the `provider` property does, albeit indirectly. The `provider` property is set to `tacocloud`, which is a reference to a separate set of configuration that describes the `tacocloud` provider's authorization server. That provider configuration is configured in the same application.yml file like this:

```
spring:
  security:
    oauth2:
      client:
...
        provider:
          tacocloud:
            issuer-uri: http://authserver:9000
```

The only property required for a provider configuration is `issuer-uri`. This property identifies the base URI for the authorization server. In this case, it refers to a server

host whose name is `authserver`. Assuming that you are running these examples locally, this is just another alias for `localhost`. On most Unix-based operating systems, this can be added in your /etc/hosts file with the following line:

```
127.0.0.1 authserver
```

Refer to documentation for your operating system for details on how to create custom host entries if /etc/hosts isn't what works on your machine.

Building on the base URL, Spring Security's OAuth 2 client will assume reasonable defaults for the authorization URL, token URL, and other authorization server specifics. But, if for some reason the authorization server you're working with differs from those default values, you can explicitly configure authorization details like this:

```
spring:
  security:
    oauth2:
      client:
        provider:
          tacocloud:
            issuer-uri: http://authserver:9000
            authorization-uri: http://authserver:9000/oauth2/authorize
            token-uri: http://authserver:9000/oauth2/token
            jwk-set-uri: http://authserver:9000/oauth2/jwks
            user-info-uri: http://authserver:9000/userinfo
            user-name-attribute: sub
```

We've seen most of these URIs, such as the authorization, token, and JWK Set URIs already. The `user-info-uri` property is new, however. This URI is used by the client to obtain essential user information, most notably the user's username. A request to that URI should return a JSON response that includes the property specified in `user-name-attribute` to identify the user. Note, however, when using Spring Authorization Server, you do not need to create the endpoint for that URI; Spring Authorization Server will expose the user-info endpoint automatically.

Now all of the pieces are in place for the application to authenticate and obtain an access token from the authorization server. Without doing anything more, you could fire up the application, make a request to any URL on that application, and be redirected to the authorization server for authorization. When the authorization server redirects back, then the inner workings of Spring Security's OAuth 2 client library will exchange the code it receives in the redirect for an access token. Now, how can we use that token?

Let's suppose that we have a service bean that interacts with the Taco Cloud API using `RestTemplate`. The following `RestIngredientService` implementation shows such a class that offers two methods: one for fetching a list of ingredients and another for saving a new ingredient:

```
package tacos;

import java.util.Arrays;
import org.springframework.web.client.RestTemplate;
```

```
public class RestIngredientService implements IngredientService {

  private RestTemplate restTemplate;

  public RestIngredientService() {
    this.restTemplate = new RestTemplate();
  }

  @Override
  public Iterable<Ingredient> findAll() {
    return Arrays.asList(restTemplate.getForObject(
            "http://localhost:8080/api/ingredients",
            Ingredient[].class));
  }

  @Override
  public Ingredient addIngredient(Ingredient ingredient) {
    return restTemplate.postForObject(
        "http://localhost:8080/api/ingredients",
        ingredient,
        Ingredient.class);
  }

}
```

The HTTP `GET` request for the /ingredients endpoint isn't secured, so the `findAll()` method should work fine, as long as the Taco Cloud API is listening on localhost, port 8080. But the `addIngredient()` method is likely to fail with an HTTP 401 response because we've secured `POST` requests to /ingredients to require `"write-Ingredients"` scope. The only way around that is to submit an access token with `"writeIngredients"` scope in the request's `Authorization` header.

Fortunately, Spring Security's OAuth 2 client should have the access token handy after completing the authorization code flow. All we need to do is make sure that the access token ends up in the request. To do that, let's change the constructor to attach a request interceptor to the `RestTemplate` it creates as follows:

```
public RestIngredientService(String accessToken) {
    this.restTemplate = new RestTemplate();
    if (accessToken != null) {
      this.restTemplate
          .getInterceptors()
          .add(getBearerTokenInterceptor(accessToken));
    }
  }
  private ClientHttpRequestInterceptor
            getBearerTokenInterceptor(String accessToken) {
    ClientHttpRequestInterceptor interceptor =
          new ClientHttpRequestInterceptor() {
      @Override
      public ClientHttpResponse intercept(
            HttpRequest request, byte[] bytes,
            ClientHttpRequestExecution execution) throws IOException {
```

```
        request.getHeaders().add("Authorization", "Bearer " + accessToken);
        return execution.execute(request, bytes);
      }
    };

    return interceptor;
  }
```

The constructor now takes a `String` parameter that is the access token. Using this token, it attaches a client request interceptor that adds the `Authorization` header to every request made by the `RestTemplate` such that the header's value is `"Bearer"` followed by the token value. In the interest of keeping the constructor tidy, the client interceptor is created in a separate `private` helper method.

Only one question remains: where does the access token come from? The following bean method is where the magic happens:

```
@Bean
@RequestScope
public IngredientService ingredientService(
              OAuth2AuthorizedClientService clientService) {
  Authentication authentication =
          SecurityContextHolder.getContext().getAuthentication();

  String accessToken = null;

  if (authentication.getClass()
            .isAssignableFrom(OAuth2AuthenticationToken.class)) {
    OAuth2AuthenticationToken oauthToken =
            (OAuth2AuthenticationToken) authentication;
    String clientRegistrationId =
            oauthToken.getAuthorizedClientRegistrationId();
    if (clientRegistrationId.equals("taco-admin-client")) {
      OAuth2AuthorizedClient client =
          clientService.loadAuthorizedClient(
              clientRegistrationId, oauthToken.getName());
      accessToken = client.getAccessToken().getTokenValue();
    }
  }
  return new RestIngredientService(accessToken);
}
```

To start, notice that the bean is declared to be request-scoped using the `@RequestScope` annotation. This means that a new instance of the bean will be created on every request. The bean must be request-scoped because it needs to pull the authentication from the `SecurityContext`, which is populated on every request by one of Spring Security's filters; there is no `SecurityContext` at application startup time when default-scoped beans are created.

Before returning a `RestIngredientService` instance, the bean method checks to see that the authentication is, in fact, implemented as `OAuth2AuthenticationToken`. If so, then that means it will have the token. It then verifies that the authentication token

is for the client named `taco-admin-client`. If so, then it extracts the token from the authorized client and passes it through the constructor for `RestIngredientService`. With that token in hand, `RestIngredientService` will have no trouble making requests to the Taco Cloud API's endpoints on behalf of the user who authorized the application.

Summary

- OAuth 2 security is a common way to secure APIs that is more robust than simple HTTP Basic authentication.
- An authorization server issues access tokens for a client to act on behalf of a user when making requests to an API (or on its own behalf in the case of client token flow).
- A resource server sits in front of an API to verify that valid, nonexpired tokens are presented with the scope necessary to access API resources.
- Spring Authorization Server is an experimental project that implements an OAuth 2 authorization server.
- Spring Security provides support for creating a resource server, as well as creating clients that obtain access tokens from the authorization server and pass those tokens when making requests through the resource server.

9 Sending messages asynchronously

This chapter covers

- Asynchronous messaging
- Sending messages with JMS, RabbitMQ, and Kafka
- Pulling messages from a broker
- Listening for messages

It's 4:55 p.m. on Friday. You're minutes away from starting a much-anticipated vacation. You have just enough time to drive to the airport and catch your flight. But before you pack up and head out, you need to be sure your boss and colleagues know the status of the work you've been doing so that on Monday they can pick up where you left off. Unfortunately, some of your colleagues have already skipped out for the weekend, and your boss is tied up in a meeting. What do you do?

The most practical way to communicate your status and still catch your plane is to send a quick email to your boss and your colleagues, detailing your progress and promising to send a postcard. You don't know where they are or when they'll read the email, but you do know they'll eventually return to their desks and read it. Meanwhile, you're on your way to the airport.

Synchronous communication, which is what we've seen with REST, has its place. But it's not the only style of interapplication communication available to developers. *Asynchronous* messaging is a way of indirectly sending messages from one application to another without waiting for a response. This indirection affords looser coupling and greater scalability between the communicating applications.

In this chapter, we're going to use asynchronous messaging to send orders from the Taco Cloud website to a separate application in the Taco Cloud kitchens where the tacos will be prepared. We'll consider three options that Spring offers for asynchronous messaging: the Java Message Service (JMS), RabbitMQ and Advanced Message Queueing Protocol (AMQP), and Apache Kafka. In addition to the basic sending and receiving of messages, we'll look at Spring's support for message-driven POJOs: a way to receive messages that resembles Enterprise JavaBeans' message-driven beans (MDBs).

9.1 Sending messages with JMS

JMS is a Java standard that defines a common API for working with message brokers. First introduced in 2001, JMS has been the go-to approach for asynchronous messaging in Java for a very long time. Before JMS, each message broker had a proprietary API, making an application's messaging code less portable between brokers. But with JMS, all compliant implementations can be worked with via a common interface in much the same way that JDBC has given relational database operations a common interface.

Spring supports JMS through a template-based abstraction known as `JmsTemplate`. Using `JmsTemplate`, it's easy to send messages across queues and topics from the producer side and to receive those messages on the consumer side. Spring also supports the notion of message-driven POJOs: simple Java objects that react to messages arriving on a queue or topic in an asynchronous fashion.

We're going to explore Spring's JMS support, including `JmsTemplate` and message-driven POJOs. Our focus will be on Spring's support for messaging with JMS, but if you want to know more about JMS, then have a look at *ActiveMQ in Action* by Bruce Snyder, Dejan Bosanac, and Rob Davies (Manning, 2011).

Before you can send and receive messages, you need a message broker that's ready to relay those messages between producers and consumers. Let's kick off our exploration of Spring JMS by setting up a message broker in Spring.

9.1.1 Setting up JMS

Before you can use JMS, you must add a JMS client to your project's build. With Spring Boot, that couldn't be any easier. All you need to do is add a starter dependency to the build. First, though, you must decide whether you're going to use Apache ActiveMQ, or the newer Apache ActiveMQ Artemis broker.

If you're using ActiveMQ, you'll need to add the following dependency to your project's pom.xml file:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-activemq</artifactId>
</dependency>
```

If ActiveMQ Artemis is the choice, the starter dependency should look like this:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-artemis</artifactId>
</dependency>
```

When using the Spring Initializr (or your IDE's frontend for the Initializr), you can also select either of these options as starter dependencies for your project. They are listed as "Spring for Apache ActiveMQ 5" and "Spring for Apache ActiveMQ Artemis," as shown in the screenshot in figure 9.1 from https://start.spring.io.

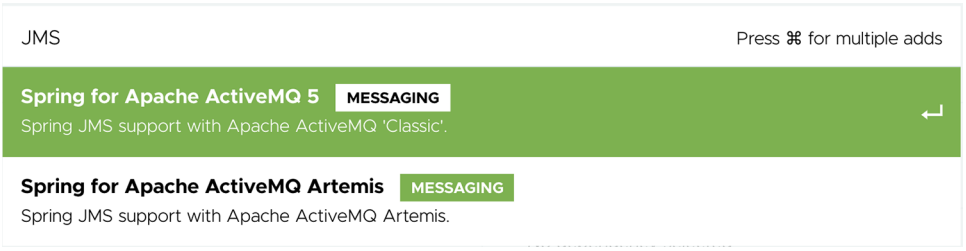

Figure 9.1 ActiveMQ and Artemis choices available in the Spring Initializr

Artemis is a next-generation reimplementation of ActiveMQ, effectively making ActiveMQ a legacy option. Therefore, for Taco Cloud you're going to choose Artemis. But the choice ultimately has little impact on how you'll write the code that sends and receives messages. The only significant differences will be in how you configure Spring to create connections to the broker.

> **RUNNING AN ARTEMIS BROKER** You'll need an Artemis broker running to be able to run the code presented in this chapter. If you don't already have an Artemis instance running, you can following the instructions from the Artemis documentation at http://mng.bz/Xr81.

By default, Spring assumes that your Artemis broker is listening on localhost at port 61616. That's fine for development purposes, but once you're ready to send your application into production, you'll need to set a few properties that tell Spring how to access the broker. The properties you'll find most useful are listed in table 9.1.

Table 9.1 Properties for configuring the location and credentials of an Artemis broker

Property	Description
spring.artemis.host	The broker's host
spring.artemis.port	The broker's port
spring.artemis.user	The user for accessing the broker (optional)
spring.artemis.password	The password for accessing the broker (optional)

For example, consider the following entry from an application.yml file that might be used in a nondevelopment setting:

```
spring:
  artemis:
    host: artemis.tacocloud.com
    port: 61617
    user: tacoweb
    password: l3tm31n
```

This sets up Spring to create broker connections to an Artemis broker listening at artemis.tacocloud.com, port 61617. It also sets the credentials for the application that will be interacting with that broker. The credentials are optional, but they're recommended for production deployments.

If you were to use ActiveMQ instead of Artemis, you'd need to use the ActiveMQ-specific properties listed in table 9.2.

Table 9.2 Properties for configuring the location and credentials of an ActiveMQ broker

Property	Description
spring.activemq.broker-url	The URL of the broker
spring.activemq.user	The user for accessing the broker (optional)
spring.activemq.password	The password for accessing the broker (optional)
spring.activemq.in-memory	Whether to start an in-memory broker (default: true)

Notice that instead of offering separate properties for the broker's hostname and port, an ActiveMQ broker's address is specified with a single property, spring.activemq.broker-url. The URL should be a tcp:// URL, as shown in the following YAML snippet:

```
spring:
  activemq:
    broker-url: tcp://activemq.tacocloud.com
    user: tacoweb
    password: l3tm31n
```